VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA" Belagavi, Karnataka – 590 018, INDIA

VTU LOGO
(insert)

MAJOR PROJECT (BCS705) Report on

"AutoDQ: A Data-Centric Framework for Evaluating Dataset Reliability and Predicting Model Performance"

Submitted in partial fulfillment of the requirements for the conferment of degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

Submitted by

[STUDENT NAME 1]	**[1BY23CSXXX]**
[STUDENT NAME 2]	**[1BY23CSXXX]**
[STUDENT NAME 3]	**[1BY23CSXXX]**
[STUDENT NAME 4]	**[1BY23CSXXX]**

Under the Guidance of

[Prof. GUIDE NAME]

[Assistant Professor], Department of CSE

BMSITM LOGO
(insert)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(An autonomous institution under VTU)

(Accredited By National Assessment & Accreditation Council (NAAC))

(Approved by AICTE, New Delhi & Affiliated to Visvesvaraya Technological University, Belagavi)

Doddaballapura Main Road, Avalahalli, Yelahanka, Bengaluru-560119.

2026-2027

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(An Autonomous Institute under VTU)

(Accredited By National Assessment & Accreditation Council (NAAC))

(Approved by AICTE, New Delhi & Affiliated to Visvesvaraya Technological University, Belagavi)

Doddaballapura Main Road, Avalahalli, Yelahanka, Bengaluru, Karnataka, INDIA – 560 119

BMSITM LOGO
(insert)

CERTIFICATE

This is to certify that the project entitled "AUTODQ: A DATA-CENTRIC FRAMEWORK FOR EVALUATING DATASET RELIABILITY AND PREDICTING MODEL PERFORMANCE" is a bonafide work carried out by [STUDENT NAME 1] ([1BY23CSXXX]), [STUDENT NAME 2] ([1BY23CSXXX]), [STUDENT NAME 3] ([1BY23CSXXX]) & [STUDENT NAME 4] ([1BY23CSXXX]) in partial fulfillment for the award of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING of the Visvesvaraya Technological University, Belagavi, during the year 2026–27. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect to work for the B.Tech degree.

Signature of the Guide
[Prof. GUIDE NAME]
[Assistant Professor], Dept. of CSE
BMSITM, Bengaluru

Associate Head
[Dr. ASSOCIATE HEAD NAME]
Professor
Dept. of CSE, BMSITM

HOD
[Dr. HOD NAME]
Professor
Dept. of CSE, BMSITM

Principal
[Dr. PRINCIPAL NAME]
Professor
BMSITM, Bengaluru

External Viva-Voce

Name of the Examiner	**Signature with Date**
1.	.....
2.	.....

INSTITUTE VISION

To emerge as one of the finest technical institutions of higher learning, to develop engineering professionals who are technically competent, ethical and environment friendly for betterment of the society.

INSTITUTE MISSION

Accomplish stimulating learning environment through high quality academic instruction, innovation and industry–institute interface.

DEPARTMENT VISION

To be a centre of excellence in Computer Science and Engineering education and research, nurturing technically competent, ethically responsible, and socially conscious professionals to meet global challenges and drive sustainable innovation.

DEPARTMENT MISSION

M1. To impart quality education in Computer Science and Engineering by integrating fundamental knowledge with emerging technologies and industry practices and research aptitude through a curriculum enriched with project-based learning, professional activities, and collaborative initiatives.
M2. To foster innovation, problem-solving, entrepreneurship and research culture through industry interaction and interdisciplinary learning.
M3. To develop graduates with strong ethical values, leadership qualities, and a commitment to lifelong learning through co-curricular and extra-curricular activities.

PROGRAM EDUCATIONAL OBJECTIVES

PEO 1 – Professional Excellence: Pursue successful careers in industry, academia, and entrepreneurship by applying the foundational knowledge of Computer Science and Engineering with professional competence.
PEO 2 – Higher Education and Lifelong Learning: Engage in higher studies, research, or professional development programs, demonstrating a commitment to lifelong learning in a rapidly evolving technological landscape.
PEO 3 – Ethics and Social Responsibility: Exhibit ethical behaviour, effective communication, teamwork, and leadership qualities, with a strong sense of responsibility toward society and the environment.

PROGRAM SPECIFIC OUTCOMES

PSO 1: Apply theoretical foundations, Algorithmic principles and software engineering practices to develop efficient and scalable IT solutions.
PSO 2: Design effective systems by leveraging principles of computing and communication technologies.

PROGRAM OUTCOMES

PO 1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO 2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO 3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO 4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)

PO 5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO 6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)

PO 7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO 8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO 9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO 10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO 11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

ACKNOWLEDGEMENT

We are happy to present this major project report after completing the work successfully. This major project would not have been possible without the guidance, assistance and suggestions of many individuals. We express our deep sense of gratitude to each and every one who has helped us to make this major project a success.

We heartily thank [Dr. PRINCIPAL NAME], Principal, BMS Institute of Technology & Management, for the constant encouragement and inspiration in taking up this major project.

We heartily thank [Dr. HOD NAME], Head of the Department of Computer Science and Engineering, BMS Institute of Technology & Management, for the constant encouragement and inspiration in taking up this major project.

We heartily thank [Dr. ASSOCIATE HEAD NAME], Associate Head, Department of Computer Science and Engineering, BMS Institute of Technology & Management, for the constant encouragement and inspiration in taking up this major project.

We gratefully thank our project guide, [Prof. GUIDE NAME], [Assistant Professor], Department of Computer Science and Engineering, for [his/her] encouragement and advice throughout the course of the major project work, and in particular for the reviews that shaped the evaluation methodology of this work.

We heartily thank the project coordinators [Dr. COORDINATOR 1 and Mrs. COORDINATOR 2], [Associate Professor / Assistant Professor], Dept. of Computer Science and Engineering, for their constant follow-up and advice throughout the major project work.

Special thanks to all the staff of the Computer Science and Engineering Department for their help, feedback and kind cooperation during the reviews and testing of the system.

Lastly, we thank our parents and friends for their encouragement and support given to us in order to finish this major project work.

[STUDENT NAME 1] ([1BY23CSXXX])
[STUDENT NAME 2] ([1BY23CSXXX])
[STUDENT NAME 3] ([1BY23CSXXX])
[STUDENT NAME 4] ([1BY23CSXXX])

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT
(An Autonomous Institution affiliated to VTU, Belagavi)
(Accredited By National Assessment & Accreditation Council (NAAC))
(Approved by AICTE, New Delhi & Affiliated to Visvesvaraya Technological University, Belagavi)
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Doddaballapura Main Road, Avalahalli, Yelahanka, Bengaluru, Karnataka, INDIA – 560 119

BMSITM LOGO
(insert)

DECLARATION

We hereby declare that the major project titled "AutoDQ: A Data-Centric Framework for Evaluating Dataset Reliability and Predicting Model Performance" is a record of original project work undertaken for partial fulfillment of the Bachelor of Technology in Computer Science and Engineering of the Visvesvaraya Technological University, Belagavi during the year 2026-2027. We have completed this major project work under the guidance of [Prof. GUIDE NAME], [Assistant Professor], Dept. of CSE.

We also declare that this major project report has not been submitted for the award of any degree, diploma, associateship, fellowship or other titles anywhere else.

[STUDENT NAME 1]	[STUDENT NAME 2]	[STUDENT NAME 3]	[STUDENT NAME 4]
USN: [1BY23CSXXX]	USN: [1BY23CSXXX]	USN: [1BY23CSXXX]	USN: [1BY23CSXXX]
Signature	Signature	Signature	Signature

ABSTRACT

The performance of a machine-learning model is bounded by the quality of the data it learns from, yet most practitioners judge a tabular dataset by eye, with ad-hoc scripts, or not at all. Missing values, duplicate records, placeholder codes such as -999 or "N/A", inconsistent category spellings, class imbalance and, most dangerously, columns that leak the target silently inflate or degrade model results. Existing data-validation tools either require writing rules in code, stop at descriptive profiling, or report problems without measuring their effect on a model.

This report presents **AutoDQ**, a data-centric web framework that evaluates the reliability of a tabular dataset and estimates how learnable it is, without the user writing any code. A dataset in CSV, Excel or JSON form is ingested, its schema is inferred, and every column is profiled. Five quality dimensions — Completeness, Consistency, Validity, Uniqueness and Balance — are scored in $[0, 1]$ and combined into an explainable, weighted **Dataset Quality Index (DQI)** on a 0–100 scale, with domain presets and a principled redistribution of weight when no target column exists. Nine issue detectors then report concrete, severity-ranked problems, including redundant features and potential target leakage tested with Benjamini–Hochberg false-discovery-rate correction, and a model-based probe that catches multivariate leakage invisible to single-column checks. Every detected issue produces a specific recommendation.

To link quality to outcomes, AutoDQ trains cross-validated baseline models (Logistic/Linear Regression and Random Forest) inside leakage-free scikit-learn pipelines, and repeats the evaluation without the columns it flagged. On a synthetic leakage dataset this exposes the problem directly: macro-F1 falls from a suspiciously perfect 1.00 to a realistic 0.91 once the leaked column is removed. A separate mutual-information diagnostic measures feature–target informativeness, and an auto-fix engine produces a corrected copy of the data together with a before/after comparison; on a deliberately damaged dataset it raised the DQI from 58.9 (Low) to 96.0 (High). Beyond analysis, the framework adds a Dataset Browser over six public catalogues, a Model Browser, and a Similar Datasets recommender that ranks real catalogue listings by structural similarity and optionally uses a large language model that receives only a privacy-safe schema summary. All outbound downloads pass through an allow-listed, size-capped guard against server-side request forgery.

The system is implemented with FastAPI, pandas, SciPy and scikit-learn on the backend and React with Vite, Tailwind CSS and Recharts on the frontend, exposing twenty REST endpoints. It is validated by 141 automated tests, by purpose-built synthetic datasets that each isolate one defect, and by a degradation harness in which DQI and cross-validated performance correlated at 0.996 and 0.683 on two datasets.

Keywords: Data Quality, Data-Centric AI, Dataset Quality Index, Data Leakage Detection, Benjamini–Hochberg Correction, Cross-Validation, Mutual Information, Class Imbalance, Automated Data Cleaning, FastAPI, React, scikit-learn

Table of Contents

Table of Contents . . . . . **iv**

List of Figures . . . . . **ix**

List of Tables . . . . . **x**

1 Introduction **1**

1.1 Background 1

1.1.1 Context and Background 1

1.1.2 Evolution of the Project 1

1.2 Overview 3

1.2.1 Scope of the Project 3

1.2.2 Applications of the Project 4

1.3 Motivation 4

1.3.1 Motivation for the Study 4

1.4 Problem Statement 5

1.4.1 Problem Identified 6

1.5 Overall Objective of the Project 6

1.5.1 Objectives of the Project 6

1.6 Summary 7

1.7 Organization of the Report 7

2 Literature Survey **8**

2.1 Review of Existing Work 8

2.1.1 Related Systems 9

2.1.2 Review of Standards and Foundational Literature 9

2.2 Comparative Analysis of Existing Systems 11

2.2.1 Comparative Review of Existing Work 11

2.3 Research Gap Analysis 15

2.3.1 Gap Analysis 15

2.4 Specific Objectives of the Project after Gap Analysis 16

2.5 Proposed Solution 16

2.6 Summary . . . 17

3 System Requirement Specification . . . 18

3.1 Overall Description . . . 18
3.1.1 Product Perspective . . . 18
3.1.2 User Classes: Major Stakeholders and Roles . . . 18
3.1.3 Operating Environment and Deployment Scenarios . . . 19
3.1.4 Operational Assumptions and Constraints . . . 19
3.2 Functional Requirements . . . 20
3.3 Non-Functional Requirements . . . 21
3.4 Hardware Requirements . . . 21
3.5 Software Requirements . . . 22
3.6 Feasibility Analysis . . . 22
3.7 Summary . . . 23

4 System Design and Architecture . . . 24

4.1 System Architecture . . . 24
4.1.1 High-level Architecture . . . 24
4.1.2 Analysis Pipeline . . . 25
4.1.3 Integrated Discovery Ecosystem . . . 26
4.1.4 Component Interaction . . . 27
4.1.5 Scalability Considerations . . . 27
4.1.6 Design Patterns Used . . . 27
4.2 Hardware and Deployment Architecture . . . 28
4.3 Software Module Design . . . 29
4.3.1 High-level Functional Modules . . . 29
4.3.2 Detailed Module Descriptions . . . 31
4.4 Security and Privacy Design . . . 33
4.4.1 Input Validation and Error Handling . . . 33
4.4.2 Server-Side Request Forgery (SSRF) Protection . . . 33
4.4.3 Secrets and Privacy . . . 34
4.5 Data Design . . . 34
4.5.1 Core Data Entities . . . 34
4.5.2 Data Storage and Consistency . . . 35
4.6 Data Flow Diagram (Level 0 and Level 1) . . . 36
4.6.1 Level 0 – Context Diagram . . . 36
4.6.2 Level 1 . . . 37
4.7 Use Case Diagram . . . 38
4.8 Activity Diagram . . . 38

4.9 Sequence Diagrams and State Transition Diagram 39
4.9.1 Upload and Analysis Sequence 39
4.9.2 Auto-fix Sequence 40
4.9.3 Similar Datasets Sequence 41
4.9.4 Dataset State Transition Diagram 41

5 System Methodology 42
5.1 Proposed Methodology 42
5.1.1 Requirement Analysis 42
5.1.2 System Design 42
5.1.3 Incremental Implementation 42
5.1.4 Testing and Validation 43
5.1.5 System Overview 44
5.1.6 Contributions 44
5.2 Algorithms 45
5.3 Explanation of the Algorithms 50
5.3.1 Robust Ingestion and Schema Inference 50
5.3.2 Target Column Auto-Detection 50
5.3.3 Validity and Consistency Scoring 50
5.3.4 Dataset Quality Index 50
5.3.5 Redundancy Detection with BH Correction 51
5.3.6 Univariate Leakage Detection and the Identifier Heuristic 51
5.3.7 Multivariate Leakage Probe 51
5.3.8 Leakage-Aware Cross-Validated Baseline 51
5.3.9 Auto-Fix 52
5.3.10 Similar Dataset Recommendation 52
5.3.11 SSRF-Safe Download 52
5.4 Pseudocode 52
5.4.1 Schema Inference 52
5.4.2 Target Auto-Detection 53
5.4.3 Dataset Quality Index 53
5.4.4 Benjamini–Hochberg Correction 54
5.4.5 Redundancy and Univariate Leakage Detection 54
5.4.6 Multivariate Leakage Probe 55
5.4.7 Leakage-Aware Baseline 55
5.4.8 Auto-Fix 56
5.4.9 Similar Dataset Scoring 56
5.4.10 SSRF-Safe Download 57

6 Implementation and Experimental Setup . . . 58
6.1 Test Cases . . . 58
6.2 Data Set . . . 59
6.3 Experimental Setup . . . 61
6.3.1 System Implementation Overview . . . 61
6.3.2 Repository Structure . . . 61
6.3.3 Running the System . . . 62
6.4 Implementation of the Modules . . . 62
6.4.1 Analysis Orchestrator . . . 62
6.4.2 DQI Engine . . . 63
6.4.3 Statistical Utilities and Shared Heuristics . . . 64
6.4.4 Baseline Model and Shared Preprocessing . . . 65
6.4.5 Auto-Fix Engine . . . 66
6.4.6 REST API . . . 66
6.4.7 Discovery and the SSRF Guard . . . 68
6.4.8 Frontend Implementation . . . 69
6.5 Description of Usage . . . 69

7 System Testing . . . 71
Testing Strategy . . . 71
7.1 Unit Testing . . . 71
7.2 Functional Testing . . . 73
7.2.1 Ingestion and Profiling . . . 73
7.2.2 Quality Metrics and DQI . . . 73
7.2.3 Issue Detection and Recommendations . . . 74
7.2.4 Baseline Model and Informativeness . . . 75
7.2.5 Auto-Fix and Report Export . . . 76
7.2.6 API and Frontend Workflow . . . 77
7.2.7 Discovery, Security and Privacy . . . 78
7.3 Integration Testing . . . 79
7.3.1 Automated Test Suite . . . 79
7.3.2 End-to-End API Integration . . . 80
7.3.3 Performance Testing . . . 80
7.4 Acceptance Testing . . . 80
7.4.1 User Acceptance Testing . . . 80
7.5 Defect Summary . . . 82

8 Results and Discussions . . . 84
8.1 Functional Achievements . . . 84

8.2 Quality Scores Across the Evaluation Datasets . . . 85
8.3 Leakage-Aware Baseline Results . . . 86
8.4 Auto-Fix Results . . . 88
8.5 Validation Harness Results . . . 89
8.6 Graphical Analysis of Test Results . . . 90
8.7 Code Base Composition . . . 90
8.8 Justification of Requirements . . . 90
8.9 Comparison with Existing Systems . . . 92
8.10 Visual Results of UI Implementation . . . 93
8.10.1 Upload and Overview . . . 94
8.10.2 Issues and Dataset Profile . . . 95
8.10.3 Model Readiness . . . 96
8.10.4 Recommendations and Auto-Fix . . . 96
8.10.5 Similar Datasets and Discovery . . . 97
8.10.6 Report Export and API Documentation . . . 98
8.11 Discussion . . . 99

Conclusion and Future Enhancements . . . 101

References . . . 103

APPENDIX A – Abbreviations . . . 106

APPENDIX B – Complex Engineering Problem . . . 108

APPENDIX C – Sustainable Development Goals Addressed . . . 112

APPENDIX D – Research Publications/Patent . . . 113

APPENDIX E – Plagiarism Report (Similarity Report and AI Detection) . . . 114

APPENDIX F – API Examples . . . 115

APPENDIX G – Local Development Setup . . . 117

APPENDIX H – Security Controls Checklist . . . 118

List of Figures

4.1 Layered architecture of AutoDQ 25
4.2 Stages of the AutoDQ analysis pipeline 26
4.3 Dataset and model discovery ecosystem around the analysis engine 26
4.4 Deployment architecture 28
4.5 Class diagram of the in-memory data model 36
4.6 Data flow diagram – Level 0 (context diagram) 37
4.7 Data flow diagram – Level 1 37
4.8 Use case diagram of AutoDQ 38
4.9 Activity diagram – analysing a dataset 39
4.10 Sequence diagram – upload and analysis 40
4.11 Sequence diagram – generating a corrected dataset 40
4.12 Sequence diagram – similar dataset recommendation 41
4.13 State transition diagram of a dataset record 41

6.1 Frontend component structure 69

8.1 DQI of each evaluation dataset with interpretation bands 86
8.2 Effect of removing flagged columns on cross-validated macro-F1 88
8.3 DQI and baseline F1 under increasing degradation 89
8.4 Test case outcomes by group 90
8.5 Lines of code per component 90
8.6 Upload page with target and domain selection 94
8.7 Overview page with DQI gauge and component breakdown 94
8.8 Issues page with severity filters 95
8.9 Dataset profile with histograms and category charts 95
8.10 Model readiness with the leakage-aware comparison 96
8.11 Recommendations and auto-fix before/after comparison 96
8.12 Similar datasets page 97
8.13 Dataset Browser 97
8.14 Model Browser 98
8.15 Exported HTML quality report 98
8.16 Interactive API documentation 99

List of Tables

1.1 Core engine (first version) versus the final AutoDQ platform 2

2.1 Standards and foundational literature applied in the design 9
2.2 Comparative review of existing work on data validation and quality tooling (Papers 1–5) 13
2.3 Comparative review of existing work on data defects, leakage and data-centric practice (Papers 6–10) 14

3.1 Stakeholders and their use of the system 18

4.1 Technology stack used for implementation 29
4.2 Functional modules of AutoDQ 30
4.3 Core data entities and their attributes 34

5.1 Implementation phases and goals 43

6.1 Test case groups and outcomes 59
6.2 Evaluation datasets and their injected defects 59
6.3 Size of the AutoDQ code base (lines, excluding dependencies) 62
6.4 DQI weights per domain preset (configurable heuristics) 64
6.5 REST API endpoints 67

7.1 Units under test and the tests that verify them 72
7.2 Ingestion and profiling test cases 73
7.3 Quality metric and DQI test cases 74
7.4 Issue detection and recommendation test cases 74
7.5 Baseline model and informativeness test cases 76
7.6 Auto-fix and report export test cases 76
7.7 API and frontend workflow test cases 77
7.8 Discovery, security and privacy test cases 78
7.9 Automated test suites 79
7.10 Pipeline execution time (2 vCPU) 80
7.11 Workflow acceptance walkthrough 81
7.12 User feedback summary (ratings out of 5) 82
7.13 Summary of failed test cases and their resolution 82

8.1 Summary of achieved functional objectives 84
8.2 DQI and component scores on the evaluation datasets (General preset) . . . 85
8.3 Best macro-F1 (5-fold CV) with all features and with flagged columns removed 87
8.4 Per-model results on dataset H (mean ± std over 5 folds) 87
8.5 Auto-fix results (re-analysed corrected datasets) 88
8.6 DQI and macro-F1 under controlled degradation 89
8.7 Justification of functional requirements 91
8.8 Justification of non-functional requirements 91
8.9 Feature comparison with existing categories of tools 92
8.10 Summary of WP competencies matched 110
8.11 Mapping of the project with UN Sustainable Development Goals 112
8.12 Mapping of OWASP API Security Top 10 (2023) risks to AutoDQ controls . 118

Chapter 1

Introduction

1.1 Background

1.1.1 Context and Background

Machine learning (ML) has moved from research laboratories into everyday decision making in healthcare, finance, retail, education and public services. Every one of these systems learns from data, and the quality of that data places an upper bound on what any model can achieve [1, 2]. A model trained on incomplete, duplicated, inconsistent, imbalanced or leaked data may still report impressive accuracy during development, yet fail silently once deployed [3].

For most of the last decade, progress in ML was measured by improvements to models: deeper networks, better optimisers and automated hyper-parameter search, evaluated on fixed benchmark datasets. The data itself was treated as a given. Industry experience has since shown that this view is incomplete. Interviews with practitioners describe *data cascades*, in which overlooked data problems compound through a pipeline and surface late as costly failures [4]. Studies of published research report that target leakage — information about the answer that should not be available at prediction time — has invalidated results across many scientific fields [5]. Even widely used benchmark test sets contain a measurable fraction of label errors [6]. These findings gave rise to *data-centric AI*, which treats the systematic engineering and assessment of data as a first-class activity [7, 8].

Despite this shift, assessing a tabular dataset before modelling remains largely manual. Practitioners write one-off scripts to count missing values, look for duplicates and plot distributions, and judge the result by experience. Mature data-validation libraries exist [9, 10], but they expect users to write rules or schemas in code and are built for production pipelines rather than for a quick, explainable verdict on an unfamiliar dataset. Few tools connect the quality of a dataset to the performance a model actually achieves on it.

1.1.2 Evolution of the Project

AutoDQ was built incrementally. A first version delivered the core analysis engine and a six-page dashboard: ingestion, profiling, five quality metrics, the Dataset Quality Index (DQI), issue detection, recommendations and a cross-validated baseline model. Subsequent phases addressed methodological weaknesses found during review and testing, added an automated

correction engine, and extended the platform with dataset and model discovery. Table 1.1 summarises how the final system extends the first version.

Table 1.1: Core engine (first version) versus the final AutoDQ platform

Aspect	**First version (core engine)**	**Final AutoDQ platform**
Quality scoring	Five metrics and a weighted DQI	Same, with domain presets, weight redistribution and per-component contributions exposed for explanation
Correlation-based detectors	Fixed thresholds ($\|r\| \geq 0.90$ / 0.95)	Thresholds plus Benjamini–Hochberg false-discovery-rate correction across all tests
Leakage detection	Univariate checks only (name, correlation, group purity, ID-like columns)	Adds a model-based probe for leakage spread across several features
Model evaluation	Single train/test split	Stratified k-fold cross-validation (mean $\pm$ std), repeated without flagged columns
Signal measurement	Not available	Mutual-information informativeness diagnostic, kept separate from DQI
Data correction	Recommendations only	Auto-fix engine producing a new corrected dataset with before/after DQI and a fix log
Validation	Synthetic datasets A–H	Adds a parameterised degradation harness measuring DQI against model performance
Data acquisition	Local upload only	Dataset Browser over six public catalogues with server-side, guarded import
Model discovery	Not available	Model Browser over Hugging Face, GitHub and Vertex AI (metadata only)
Recommendations of other data	Not available	Similar Datasets recommender with optional, privacy-preserving LLM re-ranking

Table 1.1 (continued)

Aspect	First version (core engine)	Final AutoDQ platform
Testing	39 engine tests	141 automated tests across five suites, all offline

1.2 Overview

AutoDQ is a full-stack web application that answers, for any tabular dataset, five practical questions without requiring the user to write code: *What does the data look like? How complete, consistent, valid, unique and balanced is it? Which specific problems does it contain? What should be fixed? And how well does a real baseline model learn from it?* A FastAPI backend runs a modular analysis pipeline built on pandas, NumPy, SciPy and scikit-learn; a React frontend presents the results as an overview gauge, an issues list, a column-level profile, a model-readiness page, ranked recommendations and a list of similar public datasets.

1.2.1 Scope of the Project

The scope of the project is to design, implement and validate a web-based framework that:

- ingests CSV, Excel (`.xlsx`/`.xls`) and JSON files of up to 50 MB, recovering gracefully from common formatting problems;
- infers column types and profiles every column with descriptive statistics, histograms and category frequencies;
- scores five quality dimensions and aggregates them into an explainable, domain-aware Dataset Quality Index;
- detects missing values, duplicate rows, class imbalance, outliers, redundant features, potential univariate and multivariate leakage, constant columns and dominated categorical features;
- generates one actionable recommendation for every detected issue;
- estimates dataset learnability with cross-validated baseline models, with and without suspicious columns;
- measures feature–target informativeness as a separate diagnostic;
- produces a corrected dataset and a before/after comparison on request;
- exports a printable HTML report and the complete analysis as JSON;

- lets users search and import datasets from public catalogues, browse public ML models, and discover structurally similar datasets.

Image, audio and free-text datasets, distributed processing of very large data, deep-learning baselines, user accounts with persistent storage, and protected-attribute fairness auditing are outside the present scope and are listed as future enhancements.

1.2.2 Applications of the Project

- **Pre-modelling data audits:** data scientists obtain a fast, explainable check of a new dataset before investing time in feature engineering and model tuning.
- **Detecting leakage before deployment:** the leakage-aware baseline shows when a model's apparent performance depends on a column that would not exist at prediction time.
- **Teaching and coursework:** students and instructors can see how specific data defects change a quality score and a model's cross-validated performance, using the bundled synthetic datasets.
- **Dataset selection:** researchers comparing candidate public datasets can import them from the Dataset Browser and compare their DQI and learnability on equal terms.
- **Data stewardship and governance:** teams that receive data from external sources can attach the exported report as evidence of an initial quality review.
- **Government and open data:** imports from World Bank and Open Government Data (OGD) India make the same checks available for public statistical tables.

1.3 Motivation

1.3.1 Motivation for the Study

Three observations motivated the project:

- **Data problems are common but invisible.** Missing values hidden behind placeholder strings, casing variants of the same category, repeated sentinel codes and duplicated rows rarely cause an error; they simply make a model worse, or make it look better than it is.
- **A high score is not always good news.** A model that reaches a perfect score has often learned from a leaked column. Practitioners need evidence, not only a warning, that a feature is suspicious.
- **Quality needs to be explainable and actionable.** A single opaque number is of little use; users need to see which dimension lowered the score, which columns are responsible, and what to do about it.

An automated, explainable framework that measures data quality, connects it to model behaviour and recommends concrete fixes can save analysis time, prevent misleading results and make data work accessible to users who are not expert programmers.

1.4 Problem Statement

Assessing whether a tabular dataset is reliable enough for machine learning is today a manual, inconsistent and largely unmeasured activity. Quality problems are discovered late, judged by experience rather than by stated criteria, and rarely connected to their effect on a model.

The major problems observed are:

- **Manual, repetitive checks:** every new dataset requires the same scripts for missing values, duplicates, types and distributions.
- **No single, explainable measure:** there is no agreed, traceable score that summarises dataset reliability and breaks down into its causes.
- **Hidden defects:** placeholder values, inconsistent formatting, constant columns and near-duplicate features pass unnoticed through standard loading routines.
- **Undetected leakage:** target-derived columns and combinations of features that encode the target inflate evaluation metrics and are rarely tested for.
- **Statistical false alarms:** checking hundreds of column pairs against a fixed threshold flags chance correlations as real problems.
- **No link to model outcomes:** quality reports describe the data but do not show how a model actually performs on it, or how performance changes when suspicious columns are removed.
- **Fixing is separate from finding:** even when problems are identified, producing a cleaned dataset requires further manual work.

Therefore, this project aims to develop **an automated, explainable and data-centric framework** that:

- scores dataset quality across five dimensions and summarises them in a transparent Dataset Quality Index,
- detects specific issues with stated thresholds and statistical corrections,
- demonstrates the effect of suspicious columns through a leakage-aware, cross-validated baseline,
- recommends and optionally applies corrections, and
- helps users find alternative datasets and models when the data at hand is unsuitable.

1.4.1 Problem Identified

Despite the importance of data quality, most practitioners still rely on:

- ad-hoc notebook code that differs between projects and between people,
- descriptive profiling reports that list statistics but do not grade, prioritise or explain them,
- rule-based validators that require domain rules to be written in advance,
- model metrics computed once, on a single split, with no check on why they are high.

This leads to wasted effort, inconsistent judgements and, in the worst case, models deployed on the strength of leaked features. There is no single tool that combines profiling, quantitative quality scoring, statistically corrected issue detection, model-based evidence and automated correction in an explainable, code-free interface.

1.5 Overall Objective of the Project

1.5.1 Objectives of the Project

The key objectives of the project are:

- **Define and implement an explainable Dataset Quality Index** as a weighted combination of Completeness, Consistency, Validity, Uniqueness and Balance, with configurable domain presets and transparent per-component contributions.
- **Detect concrete data issues automatically**, using stated thresholds, severity levels, Benjamini–Hochberg correction for multiple comparisons and a model-based probe for multivariate leakage.
- **Measure dataset learnability** with cross-validated baseline models trained in leakage-free pipelines, and demonstrate the effect of flagged columns by re-running the evaluation without them.
- **Separate data hygiene from predictive signal** through an independent mutual-information diagnostic.
- **Close the loop from diagnosis to action** with issue-specific recommendations and an auto-fix engine that produces a corrected dataset.
- **Make the analysis accessible** through a code-free web interface, exportable reports and a documented REST API.
- **Extend the data supply** with secure import from public catalogues, model discovery and similarity-based dataset recommendations that never expose dataset values to a third party.

- **Validate the framework** with automated tests, purpose-built defect datasets and a degradation study relating DQI to measured model performance.

These objectives together aim to reduce the effort of assessing data, prevent misleading model results and make data-centric practice routine for students, analysts and researchers.

1.6 Summary

This chapter introduced the context of data quality in machine learning and the shift towards data-centric AI, traced the evolution of AutoDQ from its core engine to the final platform, and described its scope, applications, motivation, problem statement and objectives. In short, AutoDQ replaces manual and unmeasured data checks with an automated, explainable assessment that is tied directly to model outcomes.

1.7 Organization of the Report

Chapter 2 reviews existing tools and research, compares representative work, identifies the research gap and states the specific objectives and proposed solution. Chapter 3 specifies the functional and non-functional requirements, the hardware and software requirements and the feasibility of the project. Chapter 4 presents the system architecture, module design, security and privacy design, data model and the data-flow, use-case, activity, sequence and state diagrams. Chapter 5 describes the development methodology and the algorithms used, each written as steps, explained and given as pseudocode. Chapter 6 describes the test cases, datasets, experimental setup and module implementation. Chapter 7 presents unit, functional, integration, performance and acceptance testing, and Chapter 8 the results, requirement justification and comparison with existing tools. The report then concludes with future enhancements. Appendix B maps the project to complex engineering problem attributes and Appendix C to the UN Sustainable Development Goals.

Chapter 2

Literature Survey

2.1 Review of Existing Work

Existing tools cover parts of the problem of judging whether a tabular dataset is fit for machine learning, but none combines quantitative quality scoring, statistically controlled issue detection, model-based evidence and automated correction in one explainable, code-free system. The relevant work falls into four categories: exploratory profiling tools, production data-validation frameworks, ML-oriented data quality toolkits, and research on the effect of data defects on models.

- **Exploratory profiling libraries** (for example, pandas-based profiling reports) generate per-column statistics, histograms, correlation matrices and warnings in a single HTML page. They describe a dataset well, but they do not grade it, do not prioritise problems by severity, and do not measure the consequence of a problem on a model.
- **Production data-validation frameworks** such as Amazon Deequ [9], TensorFlow Data Validation [10] and expectation-based libraries check data against constraints or an inferred schema at every pipeline run. They scale to very large data, but the constraints must be written or curated by engineers, and their output is a pass/fail list rather than an overall reliability judgement.
- **ML-oriented quality toolkits** such as the Data Linter [11] and IBM's Data Quality Toolkit [12] go further by looking for problems that specifically harm learning and by suggesting or applying remediations.
- **Empirical and conceptual research** quantifies how data defects degrade models [2], how pervasive label errors are [6], how target leakage arises [13, 5], and how neglected data work produces downstream failures [4].

AutoDQ draws on all four: it profiles like the first group, checks structural rules like the second without requiring the user to write them, targets ML-specific problems like the third, and operationalises findings of the fourth group into measurable checks with model-based evidence.

2.1.1 Related Systems

- **Data unit-testing on big-data platforms:** Deequ [9] expresses data quality as declarative constraints evaluated on Apache Spark, suggests constraints from data and detects anomalies in metric histories. It is designed for scheduled, very large-scale validation by engineering teams.
- **Schema-based validation in ML pipelines:** TensorFlow Data Validation [10] infers a schema from training data, validates every new batch against it and detects drift and training–serving skew within the TFX ecosystem.
- **Data linting:** the Data Linter [11] applies a set of lint rules to dataset summary statistics to warn about poor feature representations and to suggest transformations for a given model type.
- **Assessment-and-remediation toolkits:** the IBM Data Quality Toolkit [12] combines quality metrics, remediation functions and a data readiness report for structured data.
- **Label-error detection:** confident learning, as applied by Northcutt et al. [6], identifies likely mislabelled examples from a model's predicted probabilities.

2.1.2 Review of Standards and Foundational Literature

Because AutoDQ turns abstract quality dimensions into concrete formulas and statistical tests, its design was guided by data quality standards and by established statistical and machine-learning literature. Table 2.1 summarises these works and how each is applied.

Table 2.1: Standards and foundational literature applied in the design

Sl.	Standard / Work	Key idea	Application in AutoDQ
1	ISO/IEC 25012 data quality model [14]	Defines inherent data quality characteristics such as completeness, consistency, accuracy and credibility.	Vocabulary for the five DQI dimensions; Completeness and Consistency follow the standard's meaning.
2	Wang and Strong [15]; Pipino et al. [16]	Data quality is multi-dimensional and is assessed by simple ratios aggregated with weights.	DQI as a weighted linear sum of ratio-based component scores.
3	Batini et al. [17]	Survey of methodologies for assessing and improving data quality.	Separation of assessment (metrics, issues) from improvement (recommendations, auto-fix).
4	Shannon [18]	Entropy measures the uncertainty of a distribution.	Balance as normalised entropy of the target; normaliser for informativeness.

Table 2.1 (continued)

Sl.	Standard / Work	Key idea	Application in AutoDQ
5	Tukey [19]	Inter-quartile range fences flag unusual values.	Outlier detector (1.5×IQR) and wider (6×IQR) fence for sentinel codes.
6	Benjamini and Hochberg [20]	Controls the false discovery rate when many hypotheses are tested together.	Correlation-based redundancy and leakage findings must survive BH correction.
7	Kohavi [21]	k-fold, stratified cross-validation gives more reliable accuracy estimates than one split.	Baseline metrics reported as mean ± std over stratified folds.
8	Breiman [22]	Random forests are robust nonlinear learners with feature importances.	Baseline learner and multivariate leakage probe.
9	Kraskov et al. [23]; Ross [24]	Nearest-neighbour estimators of mutual information for continuous and mixed data.	Feature–target informativeness diagnostic (via scikit-learn).
10	Kaufman et al. [13]; Kapoor and Narayanan [5]	Formal definition and taxonomy of leakage; leakage inflates reported performance.	Five leakage checks and the with/without-flagged-columns baseline.
11	Chawla et al. [25]; He and Garcia [26]	Oversampling the minority class with synthetic examples counters imbalance.	Class-conditional resampling with jitter in the auto-fix Balance step.
12	Pedregosa et al. [27]	Composable estimators and pipelines in Python.	Pipelines refit per fold to avoid preprocessing leakage.
13	Dice [28]	Overlap coefficient between two sets.	Column-overlap component of dataset similarity.
14	OWASP API Security Top 10 and SSRF Prevention Cheat Sheet [29, 30]	Allow-lists, private-address blocking and redirect revalidation prevent server-side request forgery.	`safe_http` download guard for catalogue imports.
15	Gebru et al. [31]	Datasets should be documented for their intended use and limitations.	Exportable quality report as a lightweight, evidence-based datasheet.

2.2 Comparative Analysis of Existing Systems

2.2.1 Comparative Review of Existing Work

A review of research papers, tools and studies was carried out to understand how data quality is assessed for machine learning today. Tables 2.2 and 2.3 compare ten representative works [9, 10, 11, 12, 2, 6, 13, 5, 4, 8]. The key observations are summarised below.

1. Production Validation Frameworks (Papers 1–2) These systems treat data checks like software unit tests inside large pipelines.

- Use declarative constraints (Deequ) or an inferred, curated schema (TFDV).
- **Strengths:** scale to very large data; catch schema changes, drift and skew automatically once configured.
- **Limitations:** constraints and schemas must be authored and maintained by engineers; output is a list of violations without an overall reliability score; no measurement of the effect on model performance; no leakage analysis.

2. ML-Oriented Quality Toolkits (Papers 3–4) These works target problems that specifically affect learning.

- Apply rule sets to summary statistics (Data Linter) or compute quality metrics with paired remediations (IBM toolkit).
- **Strengths:** actionable suggestions; awareness of model-specific issues; readiness reports.
- **Limitations:** tied to specific ecosystems (TFRecord and Facets for the Data Linter; a hosted API for the IBM toolkit); no statistically corrected detection; no demonstration, for the user's own dataset, that a finding changes model performance.

3. Empirical Studies of Data Defects (Papers 5–6)

- Mohammed et al. systematically pollute datasets along six quality dimensions and measure many algorithms; Northcutt et al. estimate an average of at least 3.3% label errors across ten benchmark test sets.
- **Strengths:** rigorous evidence that data quality changes model rankings and accuracy.
- **Limitations:** offline studies; their findings are not packaged as a tool that scores a new dataset.

4. Leakage Research (Papers 7–8)

- Kaufman et al. formalise leakage; Kapoor and Narayanan show it affects at least 294 papers across 17 fields and propose model info sheets.
- **Limitations:** detection relies on expert review and checklists; no automated detector is provided.

5. Practice and Survey Literature (Papers 9–10)

- Sambasivan et al. document data cascades in high-stakes AI through practitioner interviews; Whang et al. survey data collection, cleaning and validation for deep learning.
- **Limitations:** they define the problem and the research agenda but do not provide an integrated system.

6. Common Limitations Identified Across All Work

- No single, explainable index that summarises dataset reliability and decomposes into its causes.
- Leakage detection is manual or absent; multivariate leakage is not addressed by any surveyed tool.
- Correlation-based checks use fixed thresholds without control of false discoveries.
- Quality findings are not linked to a cross-validated model on the same dataset.
- Detection and correction are separate activities; corrected data is not re-scored.
- Most tools require programming or a specific platform, which excludes non-specialist users.

Conclusion Existing work contributes valuable ideas, but none provides a comprehensive, explainable and code-free assessment that connects dataset quality to model behaviour. AutoDQ addresses this gap by integrating profiling, scoring, statistically controlled detection, leakage-aware model evaluation and automated correction in one framework.

Table 2.2: Comparative review of existing work on data validation and quality tooling (Papers 1–5)

Sl.	Paper, authors, venue, year	Methodology / technology	Measurable parameters and outcomes	Limitations	Gaps identified
1	Automating Large-Scale Data Quality Verification, S. Schelter et al., PVLDB, 2018 [9]	Declarative data unit tests on Apache Spark; constraint suggestion; incremental metrics; anomaly detection on metric history.	*Measured:* Constraint pass/fail; metric values; runtime against data size. *Outcome:* Scales validation to very large datasets; used in production pipelines.	Constraints written in code by engineers; requires Spark.	No overall quality index; no model-performance link; no leakage checks.
2	Data Validation for Machine Learning, E. Breck et al., SysML/MLSys, 2019 [10]	Schema inference and validation of each data batch; drift and training–serving skew detection in TFX.	*Measured:* Schema anomalies; distribution distance between batches. *Outcome:* Catches data errors before they reach production models.	Tied to the TensorFlow/TFX ecosystem; schemas must be curated.	No scoring or prioritisation for a single unfamiliar dataset; no leakage evidence.
3	The Data Linter: Lightweight, Automated Sanity Checking for ML Data Sets, N. Hynes, D. Sculley, M. Terry, NIPS ML Systems Workshop, 2017 [11]	Thirteen lint rules applied to summary statistics; suggests feature transforms for a model type.	*Measured:* Lints triggered per feature. *Outcome:* Surfaces poor feature encodings and suggests fixes.	Requires TFRecord and Facets summaries; recommends but does not repair.	No quality score; no statistical control; no model comparison.
4	Data Quality Toolkit, N. Gupta et al., arXiv:2108.05935, 2021 [12]	Library of quality metrics with paired remediation functions and a data readiness report.	*Measured:* Per-metric quality scores before and after remediation. *Outcome:* Detect, explain and remediate issues in structured training data.	Offered through a hosted API; assessment-centred.	No cross-validated with/without-feature comparison on the user's dataset; no multivariate leakage probe.

Table 2.2 (continued)

Sl.	Paper, authors, venue, year	Methodology / technology	Measurable parameters and outcomes	Limitations	Gaps identified
5	The Effects of Data Quality on Machine Learning Performance on Tabular Data, S. Mohammed et al., Information Systems, 2025 [2]	Systematic pollution of six quality dimensions; evaluation of classification, regression and clustering algorithms on polluted training/test data.	*Measured:* Model performance against pollution level per dimension and algorithm. *Outcome:* Quality dimensions affect algorithms differently; poor data degrades all tasks.	Offline benchmark; not a tool for assessing new data.	Findings not operationalised into a per-dataset score or detector.

Table 2.3: Comparative review of existing work on data defects, leakage and data-centric practice (Papers 6–10)

Sl.	Paper, authors, venue, year	Methodology / technology	Measurable parameters and outcomes	Limitations	Gaps identified
6	Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks, C. G. Northcutt, A. Athalye, J. Mueller, NeurIPS Datasets & Benchmarks, 2021 [6]	Confident learning flags likely label errors; candidates validated by crowd workers.	*Measured:* Estimated label-error rate per test set; change in model ranking. *Outcome:* At least 3.3% label errors on average across ten benchmarks; corrected labels change which model performs best.	Addresses label errors only; needs model probabilities.	No assessment of completeness, consistency, duplication or leakage.
7	Leakage in Data Mining: Formulation, Detection, and Avoidance, S. Kaufman et al., ACM TKDD, 2012 [13]	Formal definition of leakage via legitimacy and learn–predict separation; case studies from competitions.	*Measured:* Qualitative analysis of leaking features. *Outcome:* Clear taxonomy and avoidance methodology.	Conceptual; detection left to the analyst.	No automated detector.

Table 2.3 (continued)

Sl.	Paper, authors, venue, year	Methodology / technology	Measurable parameters and outcomes	Limitations	Gaps identified
8	Leakage and the Reproducibility Crisis in Machine-Learning-Based Science, S. Kapoor, A. Narayanan, Patterns, 2023 [5]	Survey of reviews across fields; eight-type taxonomy of leakage; model info sheets; civil-war prediction case study.	*Measured:* Number of affected papers and fields; performance before and after correcting leakage. *Outcome:* Leakage affects at least 294 papers in 17 fields; corrected complex models no longer beat simple ones.	Mitigation by manual checklists.	No tool that tests a dataset for leakage automatically.
9	"Everyone wants to do the model work, not the data work": Data Cascades in High-Stakes AI, N. Sambasivan et al., ACM CHI, 2021 [4]	Qualitative interview study of AI practitioners in high-stakes domains.	*Measured:* Occurrence and triggers of data cascades. *Outcome:* Data problems compound through pipelines and surface late.	Qualitative; no technical artefact.	Need for early, routine data assessment tooling.
10	Data Collection and Quality Challenges in Deep Learning: A Data-Centric AI Perspective, S. E. Whang et al., The VLDB Journal, 2023 [8]	Survey of data collection, cleaning, validation and robust training.	*Measured:* Not applicable (survey). *Outcome:* Consolidates techniques and open problems in data-centric AI.	No implemented system.	Integration of assessment, cleaning and evaluation left open.

2.3 Research Gap Analysis

2.3.1 Gap Analysis

From the analysis of the above work and of typical data-preparation practice, the following gaps were identified:

- Absence of a single, explainable and configurable quality index for tabular ML datasets that decomposes into per-dimension contributions.
- No automated detection of target leakage in general-purpose tools, and no detection of

leakage that arises only from combinations of features.

- Correlation-based checks that ignore the multiple-comparisons problem and therefore raise chance findings on wide datasets.
- No evidence, on the user's own dataset, of how flagged columns change cross-validated model performance.
- Data hygiene and predictive signal are not distinguished, so a clean but uninformative dataset can look excellent.
- Detection, recommendation and correction are not integrated, and corrected data is not re-assessed.
- Existing tools assume programming skill or a specific data platform.

The proposed framework addresses these gaps with an integrated pipeline that scores, detects, explains, demonstrates and corrects, behind a web interface.

2.4 Specific Objectives of the Project after Gap Analysis

Based on the gaps identified above, the specific objectives of the project are:

1. **Explainable scoring:** define a Dataset Quality Index from five ratio-based dimensions with documented formulas, domain presets and proportional weight redistribution, and expose every component's contribution.
2. **Statistically sound detection:** detect nine issue types with stated thresholds and severities, apply Benjamini–Hochberg correction to correlation tests, and add a model-based probe for multivariate leakage.
3. **Model-based evidence:** evaluate the dataset with cross-validated, leakage-free baseline pipelines and repeat the evaluation without flagged columns; report signal separately through mutual information.
4. **From finding to fixing:** map every issue to a recommendation, provide an auto-fix engine for the five DQI dimensions, and re-score the corrected data.
5. **Accessible and extensible:** deliver the system as a code-free web application with a REST API, exports, and secure discovery of alternative datasets and models.

2.5 Proposed Solution

The proposed solution, AutoDQ, is a web-based data-centric framework built on a FastAPI backend and a React frontend. An analysis pipeline ingests a dataset, infers its schema

and profiles it; five analyzers compute Completeness, Consistency, Validity, Uniqueness and Balance, which a DQI engine combines with domain-aware weights. An issue-detection engine applies nine detectors, including Benjamini–Hochberg-corrected correlation tests and a Random Forest probe for multivariate leakage, and a recommendation engine maps each issue to an action. A baseline engine trains Logistic/Linear Regression and Random Forest models under stratified cross-validation inside scikit-learn pipelines, twice when leakage or redundancy is flagged. An informativeness analyzer reports mutual information separately from the DQI, and an auto-fix engine produces a corrected dataset with a before/after comparison. Discovery services add guarded import from six public catalogues, a model browser and a similarity-based dataset recommender with optional, privacy-preserving LLM assistance.

2.6 Summary

This chapter reviewed profiling tools, data-validation frameworks, ML-oriented quality toolkits, empirical studies of data defects, leakage research and the standards that guide the design. None of the surveyed work combines explainable quality scoring, statistically controlled and multivariate issue detection, leakage-aware model evidence and automated correction in a code-free system. The specific objectives derived from this gap lead to the AutoDQ framework, whose requirements are specified in the next chapter.

Chapter 3

System Requirement Specification

This chapter specifies what AutoDQ must do and the conditions under which it operates. It describes the product and its users, lists the functional and non-functional requirements and the hardware and software needed, and analyses the feasibility of the project.

3.1 Overall Description

3.1.1 Product Perspective

AutoDQ is a new, self-contained web application for assessing tabular datasets before machine learning. It runs as two processes: a Python backend (FastAPI served by Uvicorn) that holds the analysis engine and exposes a REST API, and a single-page frontend (React, built with Vite) that runs in the browser. Datasets are held in an in-memory store for the lifetime of the backend process; no database is required.

AutoDQ interfaces with external systems only for optional features: the public APIs of Hugging Face, OpenML, Kaggle, World Bank Open Data and OGD India (dataset search and import), the Hugging Face and GitHub search APIs (model discovery), and an OpenAI-compatible large language model (LLM) endpoint offered by Groq, Google Gemini or OpenRouter (query writing and re-ranking for Similar Datasets). The core analysis never needs network access. The functional modules are described in Section 4.3.

3.1.2 User Classes: Major Stakeholders and Roles

AutoDQ is a single-user, session-based tool; it has no login or role system. Its intended users and their use of the system are listed in Table 3.1.

Table 3.1: Stakeholders and their use of the system

Stakeholder	Use of AutoDQ
Data scientist / ML engineer	Audits a new dataset before modelling; checks for leakage and redundancy; reviews the baseline learnability; downloads a corrected dataset.

Table 3.1 (continued)

Stakeholder	Use of AutoDQ
Data analyst / data steward	Produces an evidence-based quality report for data received from another team or vendor; tracks DQI across versions of a dataset.
Student / learner	Learns how specific data defects change a quality score and a model's cross-validated performance using the bundled synthetic datasets.
Researcher / academic evaluator	Compares candidate public datasets on equal terms; imports datasets from public catalogues; finds structurally similar datasets.
Instructor	Demonstrates data-centric AI concepts (imbalance, leakage, multiple comparisons) live in class.
System administrator	Installs and runs the backend and frontend, configures optional API keys and allowed origins in the `.env` file.

3.1.3 Operating Environment and Deployment Scenarios

- **Local development and demonstration (primary):** backend on `http://localhost:8000` and frontend on `http://localhost:5173` on a Windows, Linux or macOS machine; all core features work offline.
- **Shared lab server:** the same two processes on one machine, with `ALLOWED_ORIGINS` restricted to the frontend's address and optional catalogue and LLM keys configured server-side.
- **Static frontend with hosted API:** the production frontend build (`frontend/dist`) served by any static web server, pointing at the API through `VITE_API_URL`.

3.1.4 Operational Assumptions and Constraints

- Input data is tabular and fits in memory; the upload limit is 50 MB by default (`MAX_FILE_SIZE_MB`).
- Datasets and analysis results persist only while the backend runs; restarting the server clears the session history.
- The system is intended for one user or a small trusted group; it has no authentication.
- Optional API keys (Kaggle, OGD India, Hugging Face, GitHub, LLM providers) are kept in `backend/.env` and are never sent to the browser.

- The weights and interpretation bands of the DQI are heuristics, presented as configurable rather than as a validated standard.

3.2 Functional Requirements

- **FR1:** The system shall accept CSV, XLSX/XLS and JSON files up to the configured size limit, reject empty, oversized or unsupported files with a clear message, and report recoverable problems as warnings.
- **FR2:** The system shall infer the type of every column (numeric, categorical, datetime, boolean, text) and profile it with descriptive statistics, histograms or top categories.
- **FR3:** The user shall be able to select a target column and a domain preset (General, Healthcare, Finance, E-commerce); if no valid target is given, the system shall attempt to detect one.
- **FR4:** The system shall compute Completeness, Consistency, Validity, Uniqueness and Balance and combine them into a Dataset Quality Index (0–100) with an interpretation band and per-component contributions.
- **FR5:** The system shall detect missing values, duplicate rows, class imbalance, outliers, redundant features, potential leakage, potential multivariate leakage, constant columns and distributional imbalance, each with a severity, affected columns, measured value and explanation.
- **FR6:** The system shall generate one recommendation for each detected issue, ordered from critical to low severity.
- **FR7:** The system shall train cross-validated baseline models, report metrics as mean $\pm$ standard deviation, repeat the evaluation without flagged leakage/redundant columns, and assign a readiness label.
- **FR8:** The system shall compute a feature–target informativeness diagnostic, separately from the DQI.
- **FR9:** The system shall, on request, produce a corrected copy of the dataset that addresses the five DQI dimensions, re-analyse it, and return a before/after comparison, a fix log and a CSV download.
- **FR10:** The system shall export a printable HTML report and the full analysis as JSON, and list the datasets uploaded in the current session.
- **FR11:** The user shall be able to search public dataset catalogues and import a result into the analysis flow, and to search public ML models and repositories by task, library and sort order.

- **FR12:** The system shall recommend real public datasets that are structurally similar to an analysed dataset, with a similarity score, a score breakdown and a short reason.

The justification for each functional requirement is given in Section 8.8.

3.3 Non-Functional Requirements

- **Explainability:** every score must be traceable to a documented formula and the columns that drove it; heuristic weights and thresholds must be labelled as such.
- **Correctness and statistical soundness:** model evaluation must not leak information between training and test folds; multiple correlation tests must be corrected for false discoveries; leakage findings must be worded as "potential".
- **Robustness:** malformed but readable input must not crash the pipeline; edge cases (empty data, a single-class target, constant or entirely missing columns) must complete with warnings.
- **Security:** outbound downloads only from allow-listed hosts over HTTPS with size and time limits; secrets only on the server; no stack traces or keys in error messages.
- **Privacy:** no row values may be sent to an LLM; column names can be hashed on request.
- **Performance:** a dataset of a few thousand rows should be fully analysed in seconds on a laptop; result pages must read cached results instead of recomputing.
- **Usability:** the full workflow must be possible without code, with clear loading, empty and error states.
- **Maintainability and testability:** each pipeline stage must be a separate module testable without the web server; shared heuristics must live in one place.
- **Portability:** all configuration through environment variables with safe defaults; the system must run on Windows, Linux and macOS.

The justification for each non-functional requirement is given in Section 8.8.

3.4 Hardware Requirements

- **Minimum (development and demonstration):** dual-core 64-bit CPU, 4 GB RAM, about 2 GB of free disk space for the Python environment and Node.js packages. All measurements in Chapter 7 were taken on a 2-vCPU machine.
- **Recommended:** quad-core CPU and 8 GB RAM, which keeps the Random Forest cross-validation comfortably interactive for datasets of tens of thousands of rows.

- **Network:** not required for core analysis; required only for the Dataset Browser, Model Browser, Similar Datasets and LLM features.
- **Client:** any modern desktop browser (Chrome, Edge, Firefox or Safari).

3.5 Software Requirements

- Operating system: Windows 10/11 (development environment used PowerShell and VS Code), Linux or macOS
- Python 3.10 or later (the code uses `X|None` type syntax); pip and `venv`
- Backend libraries: FastAPI ($\geq$0.110), Uvicorn ($\geq$0.27), pandas ($\geq$2.0), NumPy ($\geq$1.24), SciPy ($\geq$1.10), scikit-learn ($\geq$1.3), python-multipart, openpyxl, python-dotenv
- Optional discovery libraries: httpx ($\geq$0.27), truststore ($\geq$0.9), rapidfuzz ($\geq$3.0)
- Testing: pytest ($\geq$8.0)
- Node.js 18 or later with npm
- Frontend libraries: React 18.3, Vite 5.3, Tailwind CSS 3.4, PostCSS, Autoprefixer, Recharts 2.12
- Optional services: Groq, Google Gemini or OpenRouter API key (LLM); Kaggle and OGD India API keys; Hugging Face and GitHub tokens (higher rate limits)

3.6 Feasibility Analysis

- **Technical feasibility:** every component (FastAPI, pandas, SciPy, scikit-learn, React, Vite, Recharts) is mature, well-documented open-source software. Feasibility is confirmed by the working implementation and by 141 passing automated tests.
- **Economic feasibility:** all software is free and open source; the system runs on an ordinary laptop; the optional LLM providers have free tiers, and the feature works without any key.
- **Operational feasibility:** the workflow is upload $\rightarrow$ select target $\rightarrow$ analyse $\rightarrow$ review, with no code; the bundled demo datasets and the exported report make the results easy to share.
- **Schedule feasibility:** the work was divided into phases with demonstrable exit criteria (Table 5.1), so each phase produced a usable increment.
- **Legal and ethical feasibility:** only publicly licensed catalogues are searched; nothing is scraped from sources without an API; no dataset values leave the server for the LLM; findings about imbalance and leakage are worded as potential issues and never as fairness or misconduct claims.

3.7 Summary

This chapter described the product, its users and environment, and specified twelve functional requirements, nine non-functional requirements and the hardware and software needed. The feasibility analysis shows that the project is technically, economically, operationally and legally feasible. The next chapter presents the design that satisfies these requirements.

Chapter 4

System Design and Architecture

This chapter presents the design of AutoDQ: the overall architecture and the way its components interact, the deployment structure, the software modules, the security and privacy design, the data model, and the data-flow, use-case, activity, sequence and state diagrams.

4.1 System Architecture

4.1.1 High-level Architecture

AutoDQ follows a three-layer architecture (Figure 4.1):

- **Presentation layer:** a React single-page application with three top-level tabs — *Analyze*, *Datasets* and *Models* — and six result pages inside Analyze (Overview, Issues, Dataset Profile, Model Readiness, Recommendations, Similar Datasets). Charts are drawn with Recharts; all server calls go through one small API client.
- **API layer:** a FastAPI application with four routers (datasets, discovery, models, recommender) exposing twenty REST endpoints with JSON responses, validated parameters and structured error messages.
- **Engine layer:** the analysis pipeline, the auto-fix and report services, and the discovery services. The pipeline is invoked through a single orchestrator function, `run_full_analysis`, which has no dependency on the web framework.

A process-local store keeps each uploaded or imported dataset (as a pandas DataFrame) together with its inferred schema and its most recent analysis, keyed by a generated identifier.

FIGURE TO BE INSERTED

Layered architecture. Top: Browser – React SPA (Analyze / Datasets / Models tabs; pages; Recharts charts; api.js). Middle: FastAPI app with four routers (/api/datasets, /api/discovery, /api/discovery/models, /api/datasets/{id}/alternatives + /api/llm/status) and CORS middleware. Bottom-left: Analysis engine – ingestion, profiler, five analyzers, DQI, issue detection, recommendations, baseline ML, informativeness; plus auto-fix and report export; In-memory dataset store. Bottom-right: Discovery services – dataset providers, model providers, similar-dataset recommender, LLM client, safe_http guard, TTL cache. Arrows from discovery to external APIs (Hugging Face, OpenML, Kaggle, World Bank, OGD India, GitHub, Groq/Gemini/OpenRouter).

Figure 4.1: Layered architecture of AutoDQ

4.1.2 Analysis Pipeline

The analysis pipeline is the heart of the system. It runs eight stages in a fixed order (Figure 4.2):

1. **Ingestion** — load bytes into a DataFrame, clean up headers, infer the schema.
2. **Profiling** — descriptive statistics, histograms and category frequencies.
3. **Quality metrics** — the five analyzers produce scores in $[0, 1]$ with explanations.
4. **DQI** — weighted aggregation with domain presets and weight redistribution.
5. **Issue detection** — nine detectors produce severity-ranked issues.
6. **Recommendations** — one action per issue.
7. **Baseline ML** — cross-validated models with and without flagged columns.
8. **Informativeness** — mutual information between features and target.

Target auto-detection runs before the quality metrics when no valid target was given. The orchestrator returns one result object, which the API serialises and caches on the dataset record.

FIGURE TO BE INSERTED

Left-to-right pipeline: Ingestion → Profiler → [Target auto-detect, if needed] → Five analyzers (Completeness, Consistency, Validity, Uniqueness, Balance) → DQI engine → Issue detection (9 detectors) → Recommendation engine → Baseline ML (CV, run twice if flagged) → Informativeness → AnalysisResult (cached). Show a dashed side-branch from Issue detection to Baseline ML labelled "flagged columns".

Figure 4.2: Stages of the AutoDQ analysis pipeline

4.1.3 Integrated Discovery Ecosystem

Around the analysis engine, three discovery features help users obtain better data and suitable models (Figure 4.3). The **Dataset Browser** searches six sources in parallel, including Hugging Face Datasets [32] and OpenML [33], and imports a chosen dataset through the same path as a file upload. The **Model Browser** searches public models and repositories and shows metadata only. **Similar Datasets** builds a privacy-safe schema summary of an analysed dataset, retrieves candidates from the same catalogues, scores them and optionally asks an LLM to write queries and re-rank. All outbound HTTP traffic for catalogue data passes through one guard module, `safe_http`.

FIGURE TO BE INSERTED

Centre: AutoDQ backend. Left: Dataset Browser → registry → providers (Hugging Face, OpenML, Kaggle, World Bank [local catalogue], OGD India [local catalogue], India Data Portal [link-out]) → safe_http → import → ingestion. Right: Model Browser → providers (Hugging Face models, GitHub repositories, Vertex AI link-out). Bottom: Similar Datasets → schema summary → (optional) LLM client [Groq / Gemini / OpenRouter] → query generation; registry search → scoring → (optional) LLM re-rank → ranked real listings. Mark the TTL search cache shared by all three.

Figure 4.3: Dataset and model discovery ecosystem around the analysis engine

4.1.4 Component Interaction

A typical analysis proceeds as follows:

1. The browser sends the file to `POST/api/datasets/upload`. The ingestion service validates and parses it, infers the schema and the store saves it under a new 12-character identifier. The response lists the columns and any warnings.
2. The user picks a target and domain; the browser calls `POST/api/datasets/{id}/analyze`. The orchestrator runs the pipeline and the result is cached on the record.
3. Each result page reads the already-fetched analysis object; separate `GET` endpoints (`/profile`, `/metrics`, `/issues`, ...) slice the cached result without recomputation.
4. `POST/{id}/clean` runs the auto-fix engine on the stored data, saves the corrected data as a new record, re-analyses it and returns a before/after comparison.
5. `GET/{id}/alternatives` builds the schema summary, queries the catalogues and returns ranked similar datasets; it is called only when the Similar Datasets page is opened.

4.1.5 Scalability Considerations

- **Cached analysis:** expensive work (cross-validation, the leakage probe) runs once per analyse request; result pages never trigger retraining.
- **Bounded inputs:** uploads and imports are capped by size (50 MB) and paged imports by row count (200,000); oversized imports are refused, not truncated.
- **Parallel, isolated provider calls:** catalogue searches run concurrently in worker threads with per-provider timeouts, and results are cached for 15 minutes.
- **Background catalogue sync:** catalogues without a search API (World Bank, OGD India) are synced in a background thread and searched locally; a search during sync returns a "syncing" status instead of blocking.
- **Replaceable persistence:** only `utils/store.py` knows how datasets are stored, so a database can be introduced without touching the engine.
- **Measured behaviour:** analysis time grows sub-linearly from about 5.8 s at 2,120 rows to about 14.7 s at 21,200 rows on a 2-vCPU machine (Section 7.3.3).

4.1.6 Design Patterns Used

- **Pipeline and orchestrator (facade):** `run_full_analysis` hides eight stages behind one function used by the API, the auto-fix route and the validation harness.

- **Strategy / provider interface:** every dataset source implements `DatasetProvider` and every model source `ModelProvider`; the registry treats them uniformly.
- **Registry:** provider registries map names and identifier prefixes (`hf:`, `openml:`, `kaggle:`, `wb:`, `ogd:`) to provider objects.
- **Data transfer objects:** Python dataclasses and Pydantic models (`Issue`, `DQIResult`, `DatasetCandidate`, `SchemaSummary`) define every result shape and serialise directly to JSON.
- **Adapter:** each catalogue adapter converts a provider-specific response into a common `DatasetCandidate`, and paged APIs into a CSV that the normal ingestion path accepts.
- **Fallback / graceful degradation:** the Similar Datasets service falls back from the primary LLM to a secondary one and then to deterministic keyword scoring.
- **Single source of truth for shared heuristics:** ID-column detection, preprocessing and Benjamini–Hochberg correction each live in one utility module used by every caller.

4.2 Hardware and Deployment Architecture

AutoDQ needs no special hardware. In its standard deployment (Figure 4.4) the user's browser loads the frontend from the Vite development server (or from any static server hosting the production build) and calls the FastAPI backend over HTTP. The backend reads optional keys from `backend/.env` and, for discovery features only, makes outbound HTTPS calls to allow-listed hosts.

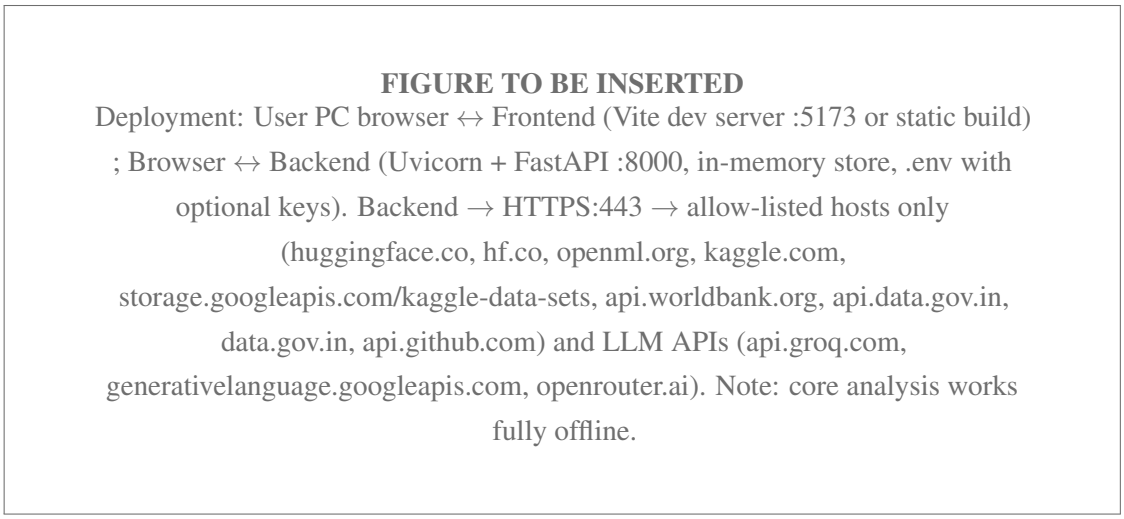


Figure 4.4: Deployment architecture

4.3 Software Module Design

4.3.1 High-level Functional Modules

The engine is built on pandas [34], NumPy [35], SciPy [36] and scikit-learn [27]; the API on FastAPI [37]; and the interface on React [38]. Table 4.1 lists the technology used in each layer and Table 4.2 the functional modules.

Table 4.1: Technology stack used for implementation

Layer	Technology	Purpose
Frontend	React 18, Vite 5	Single-page application and build tooling
Styling and charts	Tailwind CSS 3, Recharts 2	Responsive layout; DQI gauge, histograms, bar charts
API	FastAPI, Uvicorn, Pydantic	REST endpoints, validation, OpenAPI documentation at `/docs`
Data processing	pandas, NumPy	Loading, typing and transforming tabular data
Statistics	SciPy (`pearsonr`), own BH routine	Correlation p-values and false-discovery control
Machine learning	scikit-learn	Pipelines, cross-validation, Logistic/Linear Regression, Random Forest, mutual information
File formats	openpyxl, python-multipart	Excel reading; multipart upload parsing
HTTP client	httpx, truststore	Guarded outbound requests; OS certificate store
Fuzzy matching	rapidfuzz (difflib fallback)	Column-name similarity
LLM access	OpenAI-compatible REST (Groq, Gemini, OpenRouter)	Query writing and re-ranking
Configuration	python-dotenv	Environment-based settings with safe defaults
Testing	pytest	141 offline unit and integration tests

Table 4.2: Functional modules of AutoDQ

Module	Source	Responsibility
Ingestion	`services/ingestion.py`	Validate and parse CSV/Excel/JSON; schema inference
Profiler	`services/profiler.py`	Statistics, histograms, top categories
Quality analyzers	`analyzers/*.py`	Completeness, Consistency, Validity, Uniqueness, Balance
DQI engine	`metrics/dqi.py`	Weights, presets, redistribution, bands
Issue detection	`services/issue_detection.py`	Nine detectors incl. BH correction and multivariate probe
Recommendation engine	`recommendations/engine.py`	Issue-to-action mapping, severity ordering
Baseline ML	`services/baseline_ml.py`	Cross-validated models, two feature sets, readiness
Informativeness	`analyzers/informativeness.py`	Normalised mutual information per feature
Orchestrator	`services/analysis_service.py`	Target auto-detection; runs the pipeline
Auto-fix	`services/auto_fix.py`	Corrected dataset for the five DQI dimensions
Report export	`services/report_export.py`	Self-contained HTML report
Dataset discovery	`discovery/*.py`	Six providers, registry, catalogue sync, cache, `safe_http`
Model discovery	`discovery/models/*.py`	Hugging Face, GitHub, Vertex providers; allow-listed filters
Similar datasets	`services/dataset_recommender.py`, `profile_summary.py`, `llm_client.py`	Schema summary, retrieval, scoring, LLM re-ranking

Table 4.2 (continued)

Module	Source	Responsibility
Shared utilities	`utils/*.py`	ID-column heuristic, ML preprocessing, BH correction, store
Frontend	`frontend/src`	9 pages, 7 components, 4 chart components, API client

4.3.2 Detailed Module Descriptions

4.3.2.1 Ingestion Module

The ingestion module accepts raw bytes and a filename. It rejects empty files, files larger than the configured limit and unsupported extensions with an `IngestionError`. CSV and Excel files are read with pandas; JSON is accepted as a list of records, a dictionary of columns, a single object, or records nested under a `data`, `records`, `rows` or `items` key, with Latin-1 used if UTF-8 decoding fails. Fully empty "Unnamed" index columns produced by spreadsheet exports are dropped, duplicate column names are renamed, and the filename is sanitised before it is echoed. Each recovery is reported as a warning rather than an error.

Schema inference classifies each column as *boolean*, *numeric*, *datetime* (when more than 90% of a 50-value sample parses as a date and more than half matches a date pattern), *categorical* (at most $\max(20, 5\%$ of rows$)$ distinct values), *text* (high cardinality with average length above 30 characters) or *unknown* (entirely missing).

4.3.2.2 Profiler Module

For numeric columns the profiler computes count, mean, standard deviation, minimum, quartiles, maximum, skewness and kurtosis, and a histogram with $\min(20, \max(3, \lfloor\sqrt{n}\rfloor))$ bins. For categorical, boolean and text columns it reports the number of distinct values and the ten most frequent values. Columns that are entirely missing are skipped with a warning.

4.3.2.3 Quality Analyzer Modules

Each analyzer takes the DataFrame (and the schema or target where needed) and returns a result object with a score in $[0, 1]$, detailed per-column findings and a plain-language explanation. Their formulas are given in Chapter 5. Balance is reported as unavailable, not as zero, when there is no target or only one class.

4.3.2.4 DQI Engine

The DQI engine combines the five scores with the weights of the selected domain preset, redistributes the Balance weight proportionally when Balance is unavailable, and returns the overall score, the interpretation band and, for each component, its score, base weight, effective weight and contribution.

4.3.2.5 Issue Detection Module

Nine independent detector functions return `Issue` objects with a type, severity (*critical*, *high*, *medium* or *low*), affected columns, measured value and explanation. The redundancy detector and the numeric branch of the leakage detector compute p-values for every correlation and apply Benjamini–Hochberg correction. The multivariate probe runs only when the univariate checks found no leakage.

4.3.2.6 Recommendation Engine

The recommendation engine maps each issue type to a message template filled with the issue's actual column names and measured value, and sorts the result by severity. By construction, every recommendation is tied to a detected issue.

4.3.2.7 Baseline ML Module

The baseline module selects usable features (numeric, categorical and boolean columns, excluding the target and genuine identifier columns), decides between classification and regression, chooses the number of folds, and evaluates two models per task inside scikit-learn pipelines. If the issue list contains potential leakage or redundant features, it evaluates a second feature set without them and derives the readiness label from that cleaner run.

4.3.2.8 Auto-fix Module

The auto-fix module applies five ordered corrections — validity, consistency, completeness, uniqueness and balance — and records every action in a human-readable log. It never modifies the original dataset and deliberately leaves columns flagged as leaked or redundant untouched for human review.

4.3.2.9 Discovery Modules

Each dataset provider implements `search` and `fetch_for_import`; candidates carry a provider-prefixed identifier that is validated before it is used in any URL. The Model Browser

applies allow-listed task, library and sort filters and defaults to tabular tasks. The Similar Datasets service is described in Chapter 5 (Algorithm 10).

4.3.2.10 Frontend Module

The frontend keeps the Datasets and Models tabs mounted once visited so that search results survive tab switches, wraps each page in an error boundary, and shows skeleton loaders, error banners and empty states. Imported datasets re-enter the same upload $\rightarrow$ target selection $\rightarrow$ analyse flow as local files.

4.4 Security and Privacy Design

4.4.1 Input Validation and Error Handling

- Uploads are checked for emptiness, size and extension before parsing; parse failures return a short HTTP 400 message naming the error class, never a stack trace.
- Uploaded filenames are never used to build file-system paths; data is kept in memory behind a random identifier.
- Discovery parameters are validated: query length ≤ 200 characters, `limit` 1–50, `page` 1–20, and provider, task, library and sort values checked against allow-lists (unknown values return HTTP 422).
- Requests for unknown datasets return HTTP 404; requests for results before analysis return HTTP 409.

4.4.2 Server-Side Request Forgery (SSRF) Protection

Importing a dataset makes the server download from the internet, which is a classic SSRF risk [30]. The `safe_http` module therefore:

- allows only HTTPS on port 443 to an allow-list of provider hosts (with Kaggle's storage bucket allowed only under its dataset path);
- rejects IP-literal hosts and requires every resolved address to be a public (globally routable) address;
- follows redirects manually and re-checks every hop, sending authorisation headers only to the original host;
- enforces the size limit while streaming, applies per-request timeouts and an overall deadline;

- inspects zip archives in memory, rejecting path-traversal entries and capping member count and total uncompressed size.

4.4.3 Secrets and Privacy

- API keys and tokens are read from environment variables on the server only, are never returned by any endpoint, and are redacted from error messages and HTTP client logs.
- The LLM receives only a schema summary: column names and inferred types, row and column counts, target name, task type, a missing-rate bucket and the domain preset — never row values, sample rows or unique values. Setting `LLM_SEND_COLUMN_NAMES=false` replaces names with stable hashes. An automated test asserts that no cell value appears in any outgoing LLM request.
- LLM output is schema-validated and any dataset identifier not present in the retrieved candidates is discarded, so the model cannot invent recommendations.
- The frontend renders remote text as escaped plain text and opens external links only for `https://` URLs with `rel="noopenernoreferrer"`.

4.5 Data Design

4.5.1 Core Data Entities

AutoDQ does not use a database; its data model is a set of in-memory objects (Figure 4.5, Table 4.3).

Table 4.3: Core data entities and their attributes

Entity	Key attributes
StoredDataset	dataset_id, filename, df (DataFrame), schema, ingestion_warnings, last_analysis, last_target_column, last_domain, source_candidate_id
ColumnSchema	name, dtype (numeric / categorical / datetime / boolean / text / unknown), pandas_dtype, n_unique, n_missing
AnalysisResult	profile, completeness, consistency, validity, uniqueness, balance, dqi, issues, recommendations, baseline, informativeness, warnings
DQIResult	dqi_0_1, dqi_0_100, interpretation, domain_preset, weights_used, components[ ], note
Component-Contribution	name, score, weight, effective_weight, contribution, available

Table 4.3 (continued)

Entity	Key attributes
Issue	issue_type, severity, columns[ ], measured_value, explanation
Recommendation	issue_type, severity, columns[ ], message
BaselineResult	available, task_type, target_column, cv_folds, n_rows_used, full_features, cleaned_features, excluded_columns, readiness
FeatureSetResult	label, feature_columns, n_features, results[ ] (model name, metric mean and std), primary_metric_mean
FixResult	cleaned_df, fix_log[ ]
DatasetCandidate	id, provider, title, description, url, tags, license, num_rows, num_columns, importable, note, columns[ ]
SchemaSummary	columns (name, type), n_rows, n_columns, target, task_type, missing bucket, domain

4.5.2 Data Storage and Consistency

- Each dataset is stored once; analysis results are attached to its record and replaced by the next analysis.
- The auto-fix engine never mutates a dataset; the corrected data becomes a new record with its own identifier and a `_cleaned` filename suffix.
- Search results and synced catalogues live in a time-to-live cache (15 minutes for searches, 24 hours for catalogues) shared by all discovery features.

FIGURE TO BE INSERTED

UML class diagram of the in-memory data model: StoredDataset 1–* ColumnSchema; StoredDataset 1–0..1 AnalysisResult; AnalysisResult composes DatasetProfile, CompletenessResult, ConsistencyResult, ValidityResult, UniquenessResult, BalanceResult, DQIResult (1–5 ComponentContribution), Issue[*], Recommendation[*] (each Recommendation derived from exactly one Issue), BaselineResult (1–2 FeatureSetResult), InformativenessResult. StoredDataset (cleaned) –derivedFrom–> StoredDataset (original). DatasetCandidate –imported as–> StoredDataset (source_candidate_id).

Figure 4.5: Class diagram of the in-memory data model

4.6 Data Flow Diagram (Level 0 and Level 1)

4.6.1 Level 0 – Context Diagram

At Level 0 (Figure 4.6) AutoDQ is a single process. The *User* supplies a dataset file, a target column, a domain preset and search queries, and receives the quality report, issues, recommendations, model results, corrected datasets and search results. *Public dataset catalogues* supply listings and files; *Model hubs* supply model metadata; the optional *LLM service* receives schema summaries and returns queries and rankings.

FIGURE TO BE INSERTED

DFD Level 0: central process "0 AutoDQ". External entities: User (in: dataset file, target, domain, search query; out: DQI report, issues, recommendations, baseline metrics, corrected CSV, HTML/JSON report, search results), Public Dataset Catalogues (in: search/import request; out: listings, files), Model Hubs (out: model metadata), LLM Service (in: schema summary; out: queries, ranking).

Figure 4.6: Data flow diagram – Level 0 (context diagram)

4.6.2 Level 1

At Level 1 (Figure 4.7) the system decomposes into processes: 1.0 Ingest dataset, 2.0 Profile and score, 3.0 Detect issues and recommend, 4.0 Evaluate baseline, 5.0 Auto-fix, 6.0 Export report, 7.0 Discover datasets and models, 8.0 Recommend similar datasets. Two data stores are used: D1 Dataset store and D2 Search cache.

FIGURE TO BE INSERTED

DFD Level 1. Processes: 1.0 Ingest dataset (file → DataFrame + schema → D1); 2.0 Profile and score (D1 → profile, five scores, DQI); 3.0 Detect issues and recommend (D1 + scores → issues, recommendations); 4.0 Evaluate baseline (D1 + issues → CV metrics, readiness); 5.0 Auto-fix (D1 → cleaned dataset → D1, before/after DQI); 6.0 Export report (D1 analysis → HTML/JSON); 7.0 Discover datasets and models (query ↔ catalogues/model hubs via D2; import → 1.0); 8.0 Recommend similar datasets (D1 schema summary → LLM; catalogues via D2 → ranked list). Data stores: D1 Dataset store, D2 Search cache.

Figure 4.7: Data flow diagram – Level 1

4.7 Use Case Diagram

The primary actor is the *User*; secondary actors are the public catalogues, model hubs and LLM service (Figure 4.8). The main use cases are: upload dataset, import dataset from catalogue, select target and domain, analyse dataset, view overview and DQI breakdown, view issues, view profile, view model readiness, view recommendations, download corrected dataset, export report, view similar datasets, browse models and reopen a recent dataset. "Analyse dataset" includes "auto-detect target" and "run baseline"; "download corrected dataset" includes "re-analyse corrected data".

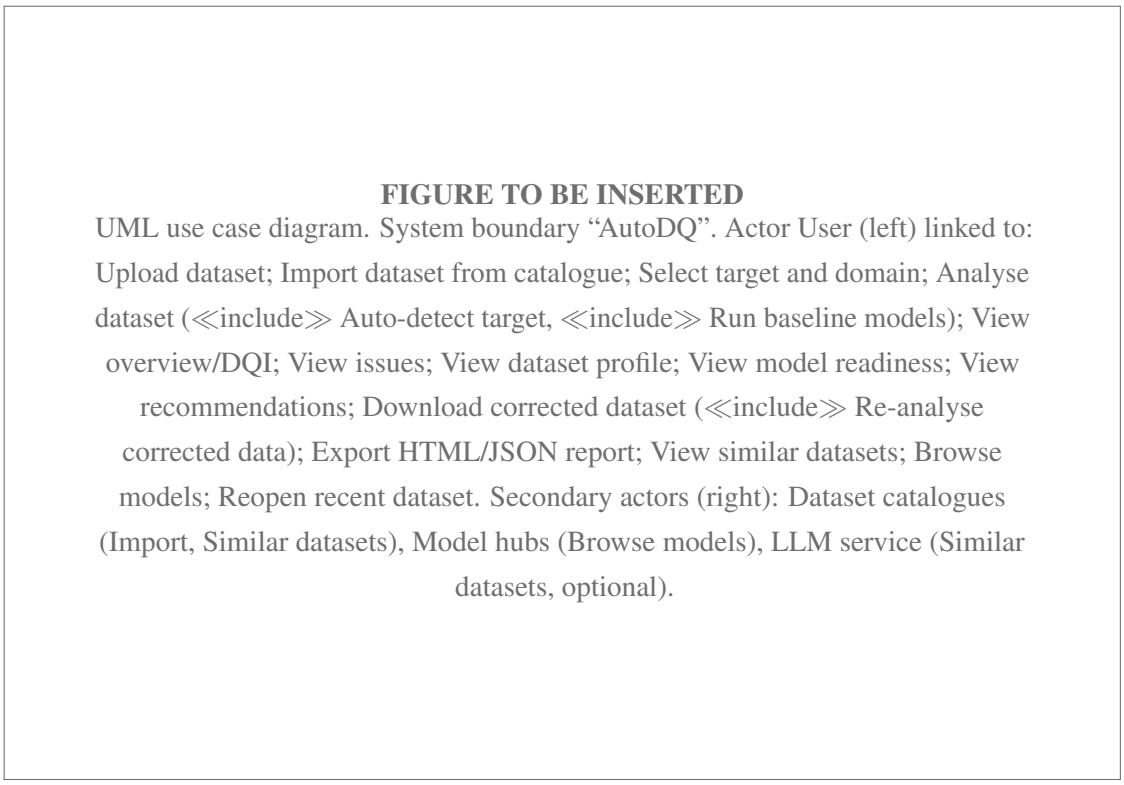


Figure 4.8: Use case diagram of AutoDQ

4.8 Activity Diagram

Figure 4.9 shows the activity of analysing a dataset: the user uploads a file; if validation fails, an error is shown; otherwise the columns are displayed, the user chooses a target and a domain (or leaves auto-detect), and starts the analysis. The pipeline runs; if a requested target does not exist, auto-detection is used and a warning is added. The result pages are shown, and the user may export a report, download a corrected dataset or open similar datasets.

FIGURE TO BE INSERTED

UML activity diagram "Analyse a dataset": Start → Upload file → [valid?] no → Show error → End; yes → Show columns and warnings → Choose target/domain → Click Analyze → [target valid?] no → Auto-detect target + warning; → Run pipeline (profile → metrics → DQI → issues → recommendations → baseline → informativeness) → Show Overview → fork: [Export report] / [Download corrected dataset → show before/after DQI] / [Open Similar Datasets] → End.

Figure 4.9: Activity diagram – analysing a dataset

4.9 Sequence Diagrams and State Transition Diagram

4.9.1 Upload and Analysis Sequence

Figure 4.10 shows the request sequence for uploading and analysing a dataset across the browser, the API router, the ingestion service, the store and the orchestrator.

FIGURE TO BE INSERTED

UML sequence: User → UploadPage: select file; UploadPage → API: POST /api/datasets/upload; API → Ingestion: load_dataset(bytes, name); Ingestion → API: df, schema, warnings; API → Store: save_dataset(); API → UploadPage: dataset_id, columns, warnings. User: choose target/domain; UploadPage → API: POST /{id}/analyze; API → Store: get_dataset(); API → Orchestrator: run_full_analysis(); Orchestrator → Profiler / Analyzers / DQI / IssueDetection / Recommender / BaselineML / Informativeness (loop of calls); Orchestrator → API: AnalysisResult; API → Store: cache last_analysis; API → UploadPage: analysis JSON; App → OverviewPage: render.

Figure 4.10: Sequence diagram – upload and analysis

4.9.2 Auto-fix Sequence

FIGURE TO BE INSERTED

UML sequence: RecommendationsPage → API: POST /{id}/clean; API → Store: get record (must be analysed, else 409); API → AutoFix: apply_fixes(df, column_types, target); AutoFix: validity → consistency → completeness → uniqueness → balance; AutoFix → API: cleaned_df, fix_log; API → Ingestion: detect_schema(cleaned_df); API → Orchestrator: run_full_analysis(cleaned); API → Store: save cleaned record; API → Page: cleaned_dataset_id, before/after DQI, fix_log, download_url; Page → API: GET /{cleaned_id}/download → CSV.

Figure 4.11: Sequence diagram – generating a corrected dataset

4.9.3 Similar Datasets Sequence

FIGURE TO BE INSERTED

UML sequence: SimilarDatasetsPage → API: GET /{id}/alternatives; API → Recommender: recommend(); Recommender → SchemaSummary: build (no cell values); [LLM configured] Recommender → LLMClient: generate 3–5 queries (fallback provider on failure; keyword queries if unavailable); loop for each (query, provider): Recommender → Registry: search (cached, parallel, per-provider timeout); Recommender: merge, de-duplicate, cap 40, score; [LLM configured] Recommender → LLMClient: re-rank top 20 and explain; drop unknown ids; Recommender → API: ranked candidates with breakdown and reason; API → Page.

Figure 4.12: Sequence diagram – similar dataset recommendation

4.9.4 Dataset State Transition Diagram

A dataset record moves through a small set of states (Figure 4.13): *Uploaded* (schema known, no analysis), *Analysed* (a cached analysis exists; re-analysis with another target or domain replaces it), and, through auto-fix, a new record in state *Cleaned-and-analysed* that is derived from it. Result endpoints are available only in the analysed states; in the *Uploaded* state they return HTTP 409.

FIGURE TO BE INSERTED

UML state diagram: [*] → Uploaded (upload or catalogue import); Uploaded → Analysed (POST analyze); Analysed → Analysed (re-analyse with new target/domain); Analysed → spawns new record “Cleaned & analysed” (POST clean); any state → [*] (server restart clears the store). Note on Uploaded: result endpoints return 409.

Figure 4.13: State transition diagram of a dataset record

Chapter 5

System Methodology

This chapter presents the development methodology, an overview of the system and its contributions, and the algorithms embedded in AutoDQ. Following the department template, each algorithm is first written as steps, then explained, and finally given as pseudocode.

5.1 Proposed Methodology

The development followed an iterative, incremental methodology. Each phase had a demonstrable goal and ended with automated tests; findings from reviews and testing fed into later phases.

5.1.1 Requirement Analysis

Requirements were derived from the project specification, from a study of how practitioners assess data before modelling, and from the literature reviewed in Chapter 2. Requirements were written as user-facing questions (“Is this dataset complete?”, “Is there leakage?”, “How well does a model learn from it?”) and as explicit constraints, for example “no recommendation may exist without a detected issue” and “no domain rule may be invented unless the user supplies it”.

5.1.2 System Design

Before implementation the pipeline order, module boundaries, API endpoints and every metric formula were fixed in design documents (architecture, DQI methodology and a mathematics reference). Synthetic datasets that each isolate one defect were designed at the same time so that every detector would have a test case before it was written.

5.1.3 Incremental Implementation

Table 5.1: Implementation phases and goals

Phase	Work carried out	Goal / exit criterion
Phase 1	Specification, architecture and DQI design documents; synthetic dataset generator (datasets A–H and the churn demo dataset).	Every planned detector has a dataset designed to trigger it.
Phase 2	Core engine: ingestion, schema inference, profiler, five analyzers, DQI, issue detection, recommendations, baseline model, orchestrator.	Engine unit tests pass on all synthetic datasets without the web server.
Phase 3	FastAPI routes, React frontend with six analysis pages, HTML/JSON report export, session history.	Full upload $\rightarrow$ analyse $\rightarrow$ review flow in the browser.
Phase 4	Methodological extensions: stratified CV, leakage-aware second baseline run, shared ID-column heuristic, Benjamini–Hochberg correction, multivariate leakage probe, informativeness diagnostic, validation harness; target auto-detection and constant-column fixes.	Engine tests pass, including BH, XOR-leakage and informativeness tests; harness reports DQI–performance correlation.
Phase 5	Auto-fix engine with before/after DQI, fix log and CSV download.	Low-quality demo dataset moves from "Low" to "High" DQI; 48 engine tests pass.
Phase 6	Dataset Browser: Hugging Face, OpenML, Kaggle; `safe_http` guard; then World Bank, OGD India and India Data Portal with background catalogue sync.	47 discovery tests pass with mocked HTTP.
Phase 7	Model Browser: Hugging Face models, GitHub repositories, Vertex AI link-out; allow-listed filters.	17 model-discovery tests pass.
Phase 8	Similar Datasets: schema summary, multi-provider LLM client with fallback, retrieval, scoring and re-ranking.	29 recommender tests pass, including the no-cell-values privacy test.

5.1.4 Testing and Validation

Each phase added tests for its own modules, and every bug found in a later phase was fixed together with a test that reproduces it. Network-dependent features are tested with mocked

HTTP transports and a mocked LLM so that the full suite runs offline. In addition to unit tests, the validation harness degrades datasets in controlled steps and measures whether DQI and model performance move together (Section 8.5).

5.1.5 System Overview

AutoDQ is a full-stack framework that turns a raw tabular file into a quality verdict, a list of concrete problems, evidence of their effect on a model, and a corrected dataset. The system comprises:

- **Frontend:** a React application with Analyze, Datasets and Models tabs, six analysis pages, charts and report export.
- **API:** a FastAPI application with twenty endpoints across four routers.
- **Analysis engine:** ingestion, profiling, five analyzers, DQI, issue detection, recommendations, baseline ML, informativeness and auto-fix.
- **Discovery services:** dataset and model providers, the similarity recommender, the LLM client and the `safe_http` guard.
- **Validation assets:** synthetic datasets, a generator, a degradation harness and 141 automated tests.

5.1.6 Contributions

The project delivers:

- An explainable Dataset Quality Index with documented formulas, domain presets, proportional weight redistribution and per-component contributions.
- Issue detectors that apply Benjamini–Hochberg false-discovery-rate correction to correlation tests, suppressing chance findings on wide datasets.
- A model-based probe that detects leakage arising only from combinations of features.
- A leakage-aware, cross-validated baseline that demonstrates the effect of flagged columns on the user's own dataset.
- A clear separation between data hygiene (DQI) and predictive signal (mutual-information informativeness).
- An auto-fix engine for the five DQI dimensions that never silently removes suspicious columns and reports what it could not fix.
- Secure dataset discovery with SSRF protection, and a similarity recommender whose optional LLM never sees data values and cannot invent results.

5.2 Algorithms

Algorithms and Rule Engines Used

This section summarises the key algorithms in AutoDQ. Each one is written below as numbered steps.

Algorithm 1: Robust Ingestion and Schema Inference

Step 1: Reject the file if it is empty, larger than the size limit, or has an unsupported extension.

Step 2: Parse CSV or Excel with pandas; for JSON, accept a list of records, a dictionary of columns, a single object, or records under a `data`, `records`, `rows` or `items` key.

Step 3: Drop entirely empty "Unnamed" columns, rename duplicate column names and record a warning for each recovery.

Step 4: For each column: if all values are missing, type it *unknown*; if boolean or numeric dtype, type it accordingly.

Step 5: For text columns, sample up to 50 values; if more than 90% parse as dates and more than half match a date pattern, type it *datetime*.

Step 6: If the distinct lower-cased values are a subset of {true, false, yes, no, 0, 1} with at most two values, type it *boolean*.

Step 7: If the number of distinct values is at most $\max(20, 0.05n)$, type it *categorical*; otherwise *text* if the average length exceeds 30 characters, else *categorical*.

Step 8: Return the DataFrame, the schema and the warnings.

Algorithm 2: Target Column Auto-Detection

Step 1: If the requested target exists, use it; otherwise record a warning and continue.

Step 2: Collect candidates: categorical or boolean columns, and numeric columns with exactly two distinct values.

Step 3: If a candidate's lower-cased name is a common target name (target, label, class, y, outcome, churn, survived, diagnosis, default, fraud), return it.

Step 4: Otherwise rank categorical candidates with 2–20 classes and binary numeric candidates by number of distinct values and return the smallest.

Step 5: If no candidate exists, return none; Balance and leakage analysis will then be limited.

Algorithm 3: Validity and Consistency Scoring

Step 1: For each numeric column, count infinite values and *sentinel* values: values outside $[Q_1 - 6IQR,\ Q_3 + 6IQR]$ that occur at least $\max(2, 0.005n)$ times. Score $= 1 -$ issues$/n$.

Step 2: For each datetime column, score $= 1 -$ unparsable$/n$.

Step 3: For each categorical, boolean or text column, score $= 1 -$ null-like tokens$/n$, where null-like tokens are {na, n/a, null, none, nan, -, –, ?, unknown, missing, n.a., nil}.

Step 4: Cap the score of any constant column at 0.8. Validity is the mean of the column scores.

Step 5: For consistency, for each categorical/boolean/text column compute raw distinct values u_r and distinct values after trimming and lower-casing u_n; score $= 1 - (u_r - u_n)/u_r$.

Step 6: For other object columns, compute the fraction f of numeric-looking values; if $0.05 < f < 0.95$, score $= 1 - 2\min(f, 1-f)$, else 1. Consistency is the mean of the column scores.

Algorithm 4: Dataset Quality Index with Weight Redistribution

Step 1: Compute Completeness $C = 1 -$ missing cells/total cells and Uniqueness $U = 1 -$ duplicate rows/rows.

Step 2: If a target with at least two classes exists, compute Balance $B = H(p)/\ln k$; otherwise mark Balance unavailable.

Step 3: Load the weights of the selected domain preset.

Step 4: If Balance is unavailable, add $w_B \cdot w_i / \sum_{j \neq B} w_j$ to each other weight w_i and set $w_B = 0$.

Step 5: Compute $DQI = \sum_i w_i s_i$, clamp to $[0, 1]$ and scale to 0–100 with one decimal.

Step 6: Assign the band: Poor (< 40), Low (< 60), Moderate (< 80) or High; return each component's score, weight, effective weight and contribution.

Algorithm 5: Redundancy Detection with Benjamini–Hochberg Correction

Step 1: Select numeric columns and drop rows with any missing value among them.

Step 2: For every pair of columns, compute the Pearson correlation r and its p-value.

Step 3: Sort the m p-values; find the largest rank k with $p_{(k)} \leq (k/m)\,\alpha$, $\alpha = 0.05$; mark ranks $1..k$ significant.

Step 4: Flag a pair only if $|r| \geq 0.90$ and it is significant; severity is high if $|r| \geq 0.97$, otherwise medium.

Algorithm 6: Univariate Leakage Detection

Step 1: For every feature whose name contains target, label, outcome, class_, y_true or prediction, report potential leakage (medium).

Step 2: If the target is numeric, compute the correlation and p-value of each numeric feature with the target, apply Benjamini–Hochberg correction, and report features with $|r| \geq 0.95$ that are significant (high).

Step 3: Otherwise, for each categorical feature with at most $0.9n$ distinct values, compute the weighted group purity $\sum_g (|g|/n) \cdot \max_c P(c \mid g)$ and report it if ≥ 0.95 (high).

Step 4: For each feature that is a genuine identifier (Algorithm 6a), report potential leakage (low).

Algorithm 6a: Identifier Column Heuristic

Step 1: If the column has fewer than $0.98n$ distinct non-missing values, it is not an identifier.

Step 2: If it is categorical or text, it is an identifier.

Step 3: If it is numeric, it is an identifier only if all values are integers and its name contains id, uuid, guid, index, _no, number or key.

Step 4: Otherwise it is not an identifier (continuous measurements are never treated as IDs).

Algorithm 7: Multivariate Leakage Probe

Step 1: Require a target, at least 40 rows with a target value and at least three usable non-identifier features.

Step 2: Split the data 70/30 (stratified for classification) and fit a 150-tree Random Forest inside the preprocessing pipeline.

Step 3: Score the held-out part with macro-F1 (classification) or R^2 (regression).

Step 4: If the score is below 0.97 (F1) or 0.95 (R^2), stop: nothing suspicious.

Step 5: If a univariate check already flagged a column, stop: that finding explains the high score.

Step 6: Otherwise report the top one to three features by importance as a potential multivariate leakage cluster.

Algorithm 8: Leakage-Aware Cross-Validated Baseline

Step 1: Drop rows without a target; require at least 30 rows.

Step 2: Select numeric, categorical and boolean features excluding the target and identifier columns.

Step 3: Choose regression if the target is numeric with more than 15 distinct values, otherwise classification.

Step 4: Set $folds = \min(5, \lfloor n/10 \rfloor)$, and for classification $folds = \min(folds, \text{smallest class})$; if $folds < 2$, report the baseline unavailable.

Step 5: Build a pipeline: numeric $\rightarrow$ median imputation and scaling; categorical $\rightarrow$ most-frequent imputation and one-hot encoding.

Step 6: Cross-validate two models (Logistic Regression and Random Forest, or Linear Regression and Random Forest Regressor) and record each metric's mean and standard deviation.

Step 7: Build the exclusion set: every column flagged for leakage, and the second column of each redundant pair.

Step 8: If the exclusion set is non-empty, repeat steps 5–6 without those columns.

Step 9: Derive readiness from the best model's primary metric in the cleaner run: F1 ≥ 0.85 High, ≥ 0.65 Moderate, else Low (R^2: 0.70 and 0.40).

Algorithm 9: Auto-Fix for the Five DQI Dimensions

Step 1: **Validity:** replace null-like tokens, repeated sentinel values and unparsable dates by missing values.

Step 2: **Consistency:** map casing/whitespace variants to the most frequent trimmed variant in their group; coerce mostly numeric mixed columns to numbers.

Step 3: **Completeness:** fill numeric columns with the median, categorical with the mode and datetime with the median date; report columns with no real values as unfixable.

Step 4: **Uniqueness:** drop exact duplicate rows.

Step 5: **Balance:** for every class below the majority count, create the deficit by sampling each column independently from that class's own values, adding Gaussian jitter $\mathcal{N}(0, (0.05\sigma)^2)$ to numeric columns.

Step 6: Leave columns flagged as leaked or redundant untouched; return the corrected data and the fix log.

Algorithm 10: Similar Dataset Recommendation

Step 1: Build a schema summary (column names and types, counts, target, task, missing-rate bucket, domain); hash names if configured.

Step 2: If an LLM is configured, ask it for 3–5 search queries; on any failure, build keyword queries from the file-name subject, the target, the top column names and the task word.

Step 3: Search every (query, provider) pair in parallel through the cached registry; merge round-robin, de-duplicate and keep at most 40 candidates; exclude link-out providers and the dataset's own source.

Step 4: Score each candidate: column overlap (Dice over fuzzy-matched names), type match, task match and size fit; drop components that cannot be computed and renormalise the weights.

Step 5: If an LLM is configured, ask it to re-rank and explain the top 20; discard any identifier not in the candidate list and add its relevance as a fifth component.

Step 6: Return candidates ordered by similarity with breakdown, coverage and reason.

Algorithm 11: SSRF-Safe Download

Step 1: Parse the URL; reject it unless the scheme is HTTPS, the port is 443, the host is not an IP literal and the host (and path, where restricted) is on the allow-list.

Step 2: Resolve the host; reject it unless every resolved address is globally routable.

Step 3: Send the request without automatic redirects, with timeouts and an overall deadline; attach credentials only for the original host.

Step 4: If the response is a redirect, validate the new URL from step 1 and repeat, up to five hops.

Step 5: Abort if `Content-Length` exceeds the limit; stream the body and abort as soon as it exceeds the limit.

Step 6: For zip archives, reject traversal paths, cap member count and uncompressed size, and keep the largest CSV/XLSX/JSON member.

5.3 Explanation of the Algorithms

5.3.1 Robust Ingestion and Schema Inference

Real files are rarely clean: spreadsheet exports add empty index columns, JSON comes in several shapes, and headers repeat. Algorithm 1 distinguishes *unrecoverable* problems, which return an error, from *recoverable* ones, which are fixed and reported. The type inference matters because every later stage depends on it: numeric columns are checked for sentinels and outliers, categorical columns for casing and placeholder tokens. The date rule requires both successful parsing and a date-like pattern, because `pandas.to_datetime` alone would accept many plain numbers as dates.

5.3.2 Target Column Auto-Detection

Balance, leakage, the baseline and informativeness all need a target. Algorithm 2 prefers well-known target names and otherwise the lowest-cardinality candidate. Including binary numeric columns was a correction made after testing on a real dataset: 0/1 targets such as `churned` are typed numeric and were previously invisible, so an unrelated categorical column was chosen and the analysis answered the wrong question.

5.3.3 Validity and Consistency Scoring

Validity uses only domain-free rules; AutoDQ deliberately does not invent constraints such as "age must be below 120". The sentinel rule requires a value to be *both* extreme (beyond six IQRs, much wider than the outlier fence) and *repeated*, because a single extreme value is more likely a genuine outlier while a value like `-999` appearing forty times is a placeholder. The consistency rule measures how many distinct values disappear after normalisation; for example {"Yes", "yes", "YES ", "No"} has four raw and two normalised values, giving $1 - 2/4 = 0.5$. The mixed-type penalty peaks at a 50/50 mix and vanishes when a column is almost uniformly one type.

5.3.4 Dataset Quality Index

The DQI is a weighted linear sum so that every point can be traced to a component; this follows the ratio-and-weights approach of the data quality literature [16]. Balance uses normalised Shannon entropy [18] because it generalises to any number of classes: a 90/10 binary target

gives $H = 0.325$ against $\ln 2 = 0.693$, so $B = 0.469$, while the user-facing "1:9" ratio is shown alongside. When no target exists, scoring Balance as 0 would unfairly penalise unsupervised data and scoring it as 1 would overstate quality; proportional redistribution keeps the other four weights in their original ratio (with defaults, Completeness rises from 0.25 to 0.3125).

5.3.5 Redundancy Detection with BH Correction

With p numeric columns there are $p(p-1)/2$ pairs, so on a wide dataset some pairs cross any fixed threshold by chance. Benjamini–Hochberg [20] controls the expected proportion of false discoveries among the flagged pairs. In the project's demonstration, 40 independent random columns with 60 rows (780 tests) produced six chance "redundant" pairs under a 0.35 threshold; BH correction suppressed all six, while genuine correlations in the test datasets were still detected.

5.3.6 Univariate Leakage Detection and the Identifier Heuristic

Leakage [13] is checked from four angles: suspicious names, near-perfect correlation with a numeric target (BH-corrected), near-perfect group purity with a categorical target, and identifier columns that let a model memorise rows. The identifier heuristic (Algorithm 6a) fixed a significant early defect: a rule based on cardinality alone classified every continuous feature (prices, sensor readings) as an identifier, so the baseline silently dropped real features and the leakage detector flagged every continuous column. Requiring integer values and an ID-like name for numeric columns, and sharing one implementation across all callers, removed the problem everywhere at once.

5.3.7 Multivariate Leakage Probe

Every univariate check looks at one feature at a time and therefore cannot see leakage that exists only in a combination. If the target is $a \oplus b$ (exclusive-or of two binary features), each feature alone is almost uncorrelated with the target — in the project's test case $r = -0.05$ and -0.10 — yet together they determine it exactly. Algorithm 7 fits a Random Forest [22] on all features and treats a near-perfect held-out score as suspicious; on the XOR case it reached F1 = 1.0 and flagged a and b together. It uses one split rather than cross-validation because it answers a yes/no question, and it stays silent when a univariate finding already explains the score.

5.3.8 Leakage-Aware Cross-Validated Baseline

Stratified k-fold cross-validation [21] gives a mean and a spread instead of a single, split-dependent number. All preprocessing sits inside a scikit-learn pipeline [27] that is refitted on each training fold, so imputation statistics and encodings never see test data. The second run

without flagged columns converts a warning into evidence: on the synthetic leakage dataset the full-feature F1 of 1.00 drops to 0.91. For a redundant pair only one column is dropped, so the model loses duplication but not signal. The readiness label is computed from the cleaner run because it is the more honest estimate.

5.3.9 Auto-Fix

The order of Algorithm 9 matters: validity cleaning turns disguised placeholders into real missing values that completeness then fills, and duplicates are removed only after normalisation so that rows differing only in spelling are recognised as duplicates. The balance step samples each column independently within a class rather than copying whole rows, which would create exact duplicates and undo the uniqueness fix. This is a lightweight approximation of synthetic oversampling [25]; unlike SMOTE it does not preserve joint relationships between columns, which the fix log states explicitly. Leaked and redundant columns are left for a human to decide.

5.3.10 Similar Dataset Recommendation

The recommender separates retrieval from judgement. Real catalogues provide the candidates; the scoring function compares structure. Column overlap uses the Dice coefficient [28] over a greedy one-to-one matching of tokenised, lightly stemmed names (exact, prefix of at least three letters, or fuzzy ratio ≥ 0.85 per token; ≥ 0.75 per name). Size fit is $1 - |\log_{10} n_c - \log_{10} n_u|/3$, floored at zero. The default weights are 0.35, 0.25, 0.20, 0.10 and 0.10 for overlap, type, task, size and LLM relevance. The LLM may improve queries and explanations but is never the source of truth: invented identifiers are dropped and every failure falls back to deterministic scoring.

5.3.11 SSRF-Safe Download

An import endpoint that downloads from a URL can be abused to reach internal services. Algorithm 11 implements the defences recommended by OWASP [30]: allow-listed hosts, public-address checks after DNS resolution, manual and re-validated redirects, streamed size limits and safe archive handling. Errors returned to the browser contain no URLs, because signed download URLs can carry tokens.

5.4 Pseudocode

5.4.1 Schema Inference

```
1 Input : column series s, row count n
```

```
Output: type in {unknown, boolean, numeric, datetime, categorical
    , text}

v <- non-missing values of s
if v is empty then return unknown
if dtype(s) is bool then return boolean
if dtype(s) is numeric then return numeric
if dtype(s) is datetime then return datetime
if dtype(s) is object then
    sample <- up to 50 values of v (fixed seed)
    if parse_rate(sample) > 0.9 and date_pattern_rate(sample) >
    0.5 then return datetime
if set(lower(v)) ⊆ {true,false,yes,no,0,1} and |unique(v)| <= 2
    then return boolean
if |unique(v)| <= max(20, 0.05 * |v|) then return categorical
if mean_length(v) > 30 then return text
return categorical
```

5.4.2 Target Auto-Detection

```
Input : DataFrame D, column types T
Output: target column or null

NAMES <- {target,label,class,y,outcome,churn,survived,diagnosis,
    default,fraud}
cat <- columns c with T[c] in {categorical, boolean}
bin <- columns c with T[c] = numeric and |unique(c)| = 2
for each c in cat + bin do
    if lower(c) in NAMES then return c
scored <- [(|unique(c)|, c) for c in cat if 2 <= |unique(c)| <=
    20] + [(2, c) for c in bin]
if scored is empty then return null
return column with the smallest |unique| in scored
```

5.4.3 Dataset Quality Index

```
Input : scores C, S, V, U, B (B may be null), domain d
Output: DQI (0-100), band, contributions

w <- PRESETS[d]                    // weights sum to 1
if B is null then
```

```
    rest <- w.C + w.S + w.V + w.U
    for each k in {C,S,V,U} do w.k <- w.k + w.B * (w.k / rest)
    w.B <- 0
q <- clamp(w.C*C + w.S*S + w.V*V + w.U*U + w.B*(B or 0), 0, 1)
dqi <- round(100 * q, 1)
band <- Poor if dqi < 40, Low if dqi < 60, Moderate if dqi < 80,
   else High
return dqi, band, {k: (score_k, base_w_k, w_k, w_k * score_k)}
```

5.4.4 Benjamini–Hochberg Correction

```
Input : p-values p[1..m], alpha = 0.05
Output: significant[1..m]

order <- indices of p sorted ascending
k <- 0
for r <- 1 to m do
    if p[order[r]] <= (r / m) * alpha then k <- r     // keep the
    LARGEST such rank
significant <- all false
for r <- 1 to k do significant[order[r]] <- true
return significant
```

5.4.5 Redundancy and Univariate Leakage Detection

```
Input : DataFrame D, types T, target y
Output: issues

N <- numeric columns; X <- rows of D[N] without missing values
pairs <- all (a, b) in N, a before b
(r, p) <- pearson(X[a], X[b]) for each pair
sig <- BH(p)
for each pair i do
    if |r_i| >= 0.90 and sig_i then add redundant(a, b, high if |
    r_i| >= 0.97 else medium)

for each feature f do
    if name(f) contains {target,label,outcome,class_,y_true,
    prediction} then add leak(f, medium)
if T[y] = numeric then
    (r, p) <- pearson(f, y) for each numeric f;  sig <- BH(p)
```

```
    for each f with |r_f| >= 0.95 and sig_f do add leak(f, high)
else
    for each categorical f with 2 <= |unique(f)| <= 0.9 n do
        purity <- sum over groups g of (|g| / n) * max_class share in g
        if purity >= 0.95 then add leak(f, high)
for each f with is_id_like(f) do add leak(f, low)
```

5.4.6 Multivariate Leakage Probe

```
Input : D, types T, target y, flagged (columns already flagged)
F <- usable non-ID features of D
if no y or rows(D) < 40 or |F| < 3 then return []
(Xtr, Xte, ytr, yte) <- split(D[F], y, test = 0.3, stratify if classification)
model <- Pipeline(preprocess(F), RandomForest(150 trees)) fitted on (Xtr, ytr)
s <- macroF1(yte, model(Xte)) or R2(yte, model(Xte))
if s < (0.97 for F1 or 0.95 for R2) then return []
if flagged is not empty then return []
top <- up to 3 features with highest importance
return [multivariate_leak(top, high, score = s)]
```

5.4.7 Leakage-Aware Baseline

```
Input : D, types T, target y, issues I
D <- rows with y present;  if |D| < 30 then return unavailable
F <- {f != y : T[f] in {numeric,categorical,boolean} and not is_id_like(f)}
task <- regression if T[y] = numeric and |unique(y)| > 15 else classification
k <- min(5, floor(|D| / 10));  if classification then k <- min(k, min_class_count)
if k < 2 then return unavailable
full <- CV(F, task, k)                                  // two models, mean and std per metric
E <- leak columns in I  ∪ {second column of each redundant pair in I}
cleaned <- CV(F \ E, task, k) if E ∩ F is non-empty else null
best <- primary metric of (cleaned or full)
readiness <- High / Moderate / Low by thresholds of task
```

```
return full, cleaned, E, readiness

function CV(F, task, k):
    pipe <- [median impute + scale (numeric) | mode impute + one-hot (categorical)] -> model
    for each model in MODELS[task] do record mean/std of metrics over k (stratified) folds
```

5.4.8 Auto-Fix

```
Input : D, types T, target y
Output: D', log

D' <- copy(D);  skip <- ID-like columns
for each column c not in skip do
    replace null-like tokens, repeated sentinels, unparsable dates in c by missing
for each categorical c do map each value to the most frequent trimmed variant of lower(trim(value))
for each mixed object c with 0.05 < numeric fraction < 0.95 do c <- to_numeric(c)
for each column c with missing values do
    if no real values remain then log "could NOT be filled" and continue
    fill with median (numeric) | mode (categorical) | median date (datetime)
D' <- drop exact duplicate rows of D'
if y exists and has >= 2 classes then
    M <- largest class count
    for each class k with count < M do
        for each column c != y do
            vals <- sample (M - count) values of c within class k, with replacement
            if c is numeric then vals <- vals + Normal(0, 0.05 * std_k(c))
        append the generated rows with y = k
return D', log
```

5.4.9 Similar Dataset Scoring

```
Input : schema summary S, candidates K, optional LLM
```

```
Q <- LLM.queries(S) if available else keyword_queries(S)
K <- dedupe(round_robin(search(q, p) for q in Q for p in
    providers))[:40]
for each c in K do
    overlap <- Dice(greedy_match(names(S), names(c)))  or  min(
    text_overlap, 0.5)
    type    <- compare type mix (and matched pairs)
    task    <- keyword / tag agreement with S.task
    size    <- max(0, 1 - |log10 rows(c) - log10 rows(S)| / 3)
    sim(c)  <- weighted mean of available components (0.35, 0.25,
    0.20, 0.10)
if LLM available then
    R <- LLM.rerank(S, top 20 of K);  drop ids not in K
    add relevance R(c) with weight 0.10 and recompute sim(c)
return K sorted by sim, with breakdown and reason
```

5.4.10 SSRF-Safe Download

```
Input : url, limit_bytes
for hop <- 1 to MAX_HOPS do
    if scheme != https or port != 443 or host is IP literal then
    reject
    if host not in ALLOWLIST (and path prefix where required)
    then reject
    if any resolved address of host is not global then reject
    resp <- GET url (no auto-redirect, timeout, auth only for
    original host)
    if resp is redirect then url <- resp.location; continue
    if Content-Length > limit_bytes then reject
    body <- stream resp; abort when size > limit_bytes
    if body is zip then check traversal, member count, total size
    ; keep largest data file
    return body
reject "too many redirects"
```

Chapter 6

Implementation and Experimental Setup

This chapter presents the implementation of the main components of AutoDQ:

- **Ingestion and orchestration** — loading, schema inference and the pipeline entry point.
- **Quality scoring and detection** — the DQI engine, Benjamini–Hochberg correction and the identifier heuristic.
- **Model evaluation and correction** — the shared preprocessing pipeline, the leakage-aware baseline and the auto-fix engine.
- **API, discovery and frontend** — REST routes, the SSRF guard, the LLM client and the React application.

Code excerpts are taken from the project repository and shortened for readability (omitted parts are marked with `...`). Following the department template, the chapter also describes the test cases and datasets used to evaluate the system, the experimental setup, and how the system is used.

6.1 Test Cases

Test cases were derived from the functional requirements (FR1–FR12), from the non-functional requirements on robustness, security and privacy, and from defects found during development and during the final review of this report. Each test case records an identifier, the scenario, the expected result, the actual result and the outcome. A case that failed when first executed and passed after a fix is marked *Fail (fixed)*; a case that still fails is marked *Fail (open)* and is tracked in the defect summary (Section 7.5). In total, 64 test cases (TC01–TC64) were defined in seven groups, summarised in Table 6.1; the full test cases are listed in Chapter 7.

Table 6.1: Test case groups and outcomes

Test case group	TC range	Cases	Passed first time	Failed, then fixed / still open
Ingestion and profiling	TC01–TC09	9	9	0 / 0
Quality metrics and DQI	TC10–TC18	9	9	0 / 0
Issue detection and recommendations	TC19–TC32	14	10	2 / 2
Baseline model and informativeness	TC33–TC41	9	7	1 / 1
Auto-fix and report export	TC42–TC49	8	6	1 / 1
API and frontend workflow	TC50–TC54	5	4	1 / 0
Discovery, security and privacy	TC55–TC64	10	10	0 / 0
Total		**64**	**55**	**5 / 4**

In addition to the documented cases, 141 automated tests in five pytest suites run offline against the engine, the API and mocked external services (Section 7.3.1).

6.2 Data Set

AutoDQ is an analysis framework rather than a trained model, so there is no training dataset in the usual sense. Instead, the system is evaluated on datasets whose defects are known in advance, so that every detector can be checked against ground truth. All of them are generated reproducibly by `datasets/generate_datasets.py` with fixed random seeds.

Table 6.2: Evaluation datasets and their injected defects

Dataset	Rows × cols	Designed problem	Injection method
`A_clean.csv`	1000 × 4	None (baseline)	Two numeric features, one categorical feature, balanced binary target with a learnable signal
`B_missing_values.csv`	1000 × 4	Missing values	15% of `feature_1` and 30% of `category` set to missing
`C_class_imbalance.csv`	555 × 4	Class imbalance (≈1:9)	Minority class down-sampled to 11%

Table 6.2 (continued)

Dataset	Rows × cols	Designed problem	Injection method
D_duplicates.csv	1008 × 4	Duplicate rows	Rows re-appended as exact copies
E_outliers.csv	1000 × 4	Statistical outliers	5% of feature_1 multiplied by ±8
F_redundant_features.csv	1000 × 5	Redundant feature	feature_1_copy = feature_1 × 1.02 + small noise
G_leakage.csv	1000 × 5	Target leakage	target_echo encodes the target (0.01/0.99 + noise)
H_combined_problems.csv	621 × 6	All of the above	Imbalance, missing values, outliers, redundancy, leakage and duplicates stacked
demo_customer_churn.csv	2120 × 10	Realistic mixture	Churn driven by contract, tenure and support calls; missing values (≈8% monthly_charges, ≈5% contract_type); 6% re-appended rows; eight impossible ages (150, −5, 999); estimated_annual_value ≈ 12 × monthly_charges; retention_flag = 1 − churn
low_quality_demo.csv	850 × 10	Heavy damage	Very high missingness, ≈47% duplicate rows, ≈1:52 imbalance, inconsistent casing, placeholder text, duplicated columns
high_readiness_demo.csv	1000 × 5	None (positive control)	Clean student-performance data with a learnable passed target

Observation on the churn dataset. The generator adds random noise to estimated_annual_value *after* re-appending the 6% duplicate rows, so 108 of the 120 duplicates differ in that column and only 12 remain exact duplicates (0.57%). Without that column, 119 rows are duplicates. AutoDQ therefore reports 0.57% exact duplication on this dataset; the remaining

near-duplicates are not detected because `customer_id` is not recognised as an identifier (Section 7.5).

Additional data sources. For discovery features, real datasets can be imported from Hugging Face, OpenML, Kaggle, World Bank Open Data and OGD India. Automated tests for these sources use recorded, mocked responses so that results are repeatable and no network access is required.

Validation harness. `datasets/validation_harness.py` degrades any dataset at five intensity levels (0.0 to 1.0) by injecting missing values, duplicate rows and outliers (outliers only into columns with more than two distinct values), runs the full pipeline at each level and reports the Pearson correlation between the DQI and the baseline's primary metric.

6.3 Experimental Setup

6.3.1 System Implementation Overview

The experiments use the hardware and software listed in Sections 3.4 and 3.5. The development environment was Windows with VS Code and PowerShell, a Python virtual environment for the backend and Node.js for the frontend. All results reported in Chapters 7 and 8 were re-run for this report on a 2-vCPU Linux machine with Python 3.13, pandas 3.0.5, NumPy 2.5.3, SciPy 1.18.1 and scikit-learn 1.9.1. All randomness is seeded (`random_state=42` for cross-validation and models, fixed seeds in the generators), so results are reproducible.

6.3.2 Repository Structure

```
autodq/
  backend/
    app/
      main.py                 FastAPI app, CORS, router registration
      api/                    routes.py, discovery_routes.py, models_routes.py,
                              recommender_routes.py
      analyzers/              completeness, consistency, validity, uniqueness,
                              balance, informativeness
      metrics/dqi.py          DQI weights, presets, redistribution, bands
      recommendations/        engine.py (issue -> recommendation)
      services/               ingestion, profiler, issue_detection, baseline_ml,
                              analysis_service, auto_fix, report_export,
                              dataset_recommender, profile_summary, llm_client
      discovery/              huggingface, openml, kaggle, worldbank, ogd_india,
                              india_data_portal, catalog_sync, cache, registry,
                              safe_http, models/ (huggingface, github, vertex)
      utils/                  column_heuristics, ml_preprocessing, stats_utils, store
    tests/                    5 pytest suites (141 tests)
    requirements.txt, requirements-discovery.txt, .env.example
  frontend/
```

```
  src/  App.jsx, pages/ (9), components/ (7), charts/ (4), services/api.js
  package.json, vite.config.js, tailwind.config.js, .env.example
datasets/  generate_datasets.py, validation_harness.py, 11 CSV datasets
docs/      architecture, dqi, math_and_logic, testing,
           limitations_and_future_validation, feature_roadmap
```

Table 6.3: Size of the AutoDQ code base (lines, excluding dependencies)

Component	Files	Lines
Backend services (`services/`)	11	2,754
Discovery and model discovery (`discovery/`)	19	2,180
Quality analyzers (`analyzers/`)	7	670
API routes (`api/`)	5	437
Utilities, DQI, recommendations, schemas	11	461
Automated tests (`tests/`)	6	2,381
Frontend pages, components, charts, API client	21	1,942
Dataset generator and validation harness	2	310
Technical documentation (`docs/`)	6	1,610

6.3.3 Running the System

The backend is started with `python-muvicornapp.main:app--reload` (API on port 8000, interactive documentation at `/docs`); the frontend with `npmrundev` (port 5173). Configuration is read from `backend/.env` and `frontend/.env`; every setting has a safe default, so a fresh checkout runs without either file. The complete setup procedure is given in Appendix G.

6.4 Implementation of the Modules

6.4.1 Analysis Orchestrator

The orchestrator is the single entry point used by the API, the auto-fix route and the validation harness. It resolves the target, runs every stage in order and passes the issue list to the baseline so that flagged columns can be excluded.

Listing 6.1: Pipeline orchestration (`services/analysis_service.py`)

```
1 def run_full_analysis(df, schema, target_column=None, domain="general",
```

```
                        ingestion_warnings=None) -> AnalysisResult:
    column_types = {s.name: s.dtype for s in schema}
    warnings = list(ingestion_warnings or [])
    profile = build_profile(df, schema)
    if target_column and target_column not in df.columns:
        warnings.append(f"Requested target column '{target_column}' not
    found; "
                        "falling back to auto-detection.")
        target_column = None
    if not target_column:
        target_column = detect_default_target(df, column_types)
    completeness = analyze_completeness(df)
    consistency  = analyze_consistency(df, column_types)
    validity     = analyze_validity(df, column_types)
    uniqueness   = analyze_uniqueness(df, column_types)
    balance      = analyze_balance(df, target_column)
    dqi = compute_dqi(completeness.score, consistency.score,
    validity.score,
                      uniqueness.score,
                      balance.score if balance.available else None,
    domain=domain)
    issues = detect_all_issues(df, column_types, target_column,
                  class_counts=balance.class_counts if balance.available
    else None)
    recommendations = generate_recommendations(issues)
    baseline = run_baseline(df, target_column, column_types,
    issues=issues)
    informativeness = analyze_informativeness(df, column_types,
    target_column)
    return AnalysisResult(profile, completeness, ..., informativeness,
    warnings)
```

6.4.2 DQI Engine

Listing 6.2: Weight redistribution when Balance is unavailable (`metrics/dqi.py`)

```
DOMAIN_PRESETS = {
  "general":    {"completeness": .25, "consistency": .20, "validity":
    .20,
                 "uniqueness": .15, "balance": .20},
  "healthcare": {"completeness": .30, "consistency": .20, "validity":
    .20,
                 "uniqueness": .10, "balance": .20},
  ... }

effective_weights = dict(weights)
```

```
if balance is None:
    redistribute = weights["balance"]
    remaining = [k for k in weights if k != "balance"]
    remaining_total = sum(weights[k] for k in remaining)
    for k in remaining:
        effective_weights[k] = weights[k] + redistribute * weights[k] /
    remaining_total
    effective_weights["balance"] = 0.0
```

Table 6.4 lists the four domain presets.

Table 6.4: DQI weights per domain preset (configurable heuristics)

Preset	Completeness	Consistency	Validity	Uniqueness	Balance
General (default)	0.25	0.20	0.20	0.15	0.20
Healthcare	0.30	0.20	0.20	0.10	0.20
Finance	0.20	0.30	0.20	0.10	0.20
E-commerce	0.25	0.20	0.15	0.20	0.20

6.4.3 Statistical Utilities and Shared Heuristics

The Benjamini–Hochberg routine is a dependency-free implementation used by both the redundancy and the numeric-leakage detectors.

Listing 6.3: Benjamini–Hochberg step-up procedure (`utils/stats_utils.py`)

```
def benjamini_hochberg(p_values, alpha=0.05):
    m = len(p_values)
    indexed = sorted(range(m), key=lambda i: p_values[i])
    threshold_rank = -1
    for rank, idx in enumerate(indexed, start=1):
        if p_values[idx] <= (rank / m) * alpha:
            threshold_rank = rank   # BH takes the LARGEST qualifying
    rank
    significant = [False] * m
    for rank, idx in enumerate(indexed, start=1):
        if rank <= threshold_rank:
            significant[idx] = True
    return significant
```

Listing 6.4: Shared identifier heuristic (`utils/column_heuristics.py`)

```
ID_LIKE_NAME_TOKENS = ("id", "uuid", "guid", "index", "_no", "number",
    "key")

def is_id_like_column(name, series, column_type, n_rows) -> bool:
```

```
    if n_rows == 0:
        return False
    if series.nunique(dropna=True) < 0.98 * n_rows:
        return False                        # not high-cardinality
    if column_type in ("categorical", "boolean", "text"):
        return True                         # free text / generated
    string id
    ...  # numeric: integer-valued AND name contains an ID-like token
```

6.4.4 Baseline Model and Shared Preprocessing

Listing 6.5: Preprocessing inside the cross-validated pipeline (`utils/ml_preprocessing.py`)

```
def build_preprocessor(feature_cols, column_types) -> ColumnTransformer:
    numeric_cols = [c for c in feature_cols if column_types.get(c) ==
    "numeric"]
    categorical_cols = [c for c in feature_cols
                        if column_types.get(c) in ("categorical",
    "boolean")]
    transformers = []
    if numeric_cols:
        transformers.append(("num", Pipeline([
            ("impute", SimpleImputer(strategy="median")),
            ("scale", StandardScaler())]), numeric_cols))
    if categorical_cols:
        transformers.append(("cat", Pipeline([
            ("impute", SimpleImputer(strategy="most_frequent")),
            ("encode", OneHotEncoder(handle_unknown="ignore"))]),
    categorical_cols))
    return ColumnTransformer(transformers, remainder="drop")
```

Listing 6.6: Choosing the columns for the second, cleaner run (`services/baseline_ml.py`)

```
def _determine_leakage_and_redundancy_exclusions(issues, target_column):
    excluded = set()
    for issue in issues:
        if issue.issue_type in ("potential_leakage",
    "potential_multivariate_leakage"):
            excluded.update(c for c in issue.columns if c !=
    target_column)
        elif issue.issue_type == "redundant_features" and
    len(issue.columns) == 2:
            a, b = issue.columns
            if a not in excluded and b not in excluded:
                excluded.add(b)          # keep a, drop b
    return sorted(excluded)
...
```

```
cv = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=42)
scoring = {"accuracy": "accuracy", "precision": "precision_macro",
           "recall": "recall_macro", "f1": "f1_macro"}
for name, model in models.items():
    pipe = Pipeline([("prep", preprocessor), ("model", model)])
    scores = cross_validate(pipe, X, y, cv=cv, scoring=scoring)
```

6.4.5 Auto-Fix Engine

Listing 6.7: Class-conditional resampling with jitter (`services/auto_fix.py`)

```
for cls, count in counts.items():
    deficit = majority_count - int(count)
    if deficit <= 0:
        continue
    class_subset = df[df[target_column] == cls]
    synthetic_cols = {}
    for col in df.columns:
        if col == target_column:
            synthetic_cols[col] = [cls] * deficit
            continue
        sampled = class_subset[col].dropna()
        values = sampled.sample(n=deficit, replace=True,
                    random_state=int(rng.integers(0,
    1_000_000))).to_numpy()
        if column_types.get(col) == "numeric" and col not in id_like:
            std = float(np.nanstd(sampled.to_numpy(dtype=float)))
            if std > 0:
                values = values.astype(float) + rng.normal(0, 0.05 *
    std, size=deficit)
        synthetic_cols[col] = values
    synthetic_frames.append(pd.DataFrame(synthetic_cols))
```

6.4.6 REST API

Listing 6.8: Analyse and clean endpoints (`api/routes.py`, shortened)

```
@router.post("/{dataset_id}/analyze")
def analyze_dataset(dataset_id: str, request: AnalyzeRequest):
    record = get_dataset(dataset_id)
    if not record:
        raise HTTPException(status_code=404, detail="Dataset not
    found.")
    result = run_full_analysis(record.df, record.schema,
                               target_column=request.target_column,
                               domain=request.domain,
```

```
9
      ingestion_warnings=record.ingestion_warnings)
10     record.last_analysis = analysis_result_to_dict(result)
11     ...
12 @router.post("/{dataset_id}/clean")
13 def clean_dataset(dataset_id: str):
14     ...
15     fix_result = apply_fixes(record.df, column_types, target_column)
16     after = run_full_analysis(fix_result.cleaned_df,
17                               detect_schema(fix_result.cleaned_df), ...)
18     cleaned = save_dataset(_derive_cleaned_filename(record.filename),
19                            fix_result.cleaned_df, cleaned_schema, [])
20     return {"cleaned_dataset_id": cleaned.dataset_id, "fix_log":
      fix_result.fix_log,
21             "before": {...}, "after": {...},
22             "download_url":
      f"/api/datasets/{cleaned.dataset_id}/download"}
```

Table 6.5 lists all endpoints.

Table 6.5: REST API endpoints

Method	Endpoint	Purpose
POST	`/api/datasets/upload`	Upload and parse a file
GET	`/api/datasets`	Session history with DQI of analysed datasets
GET	`/api/datasets/{id}`	Dataset metadata and schema
POST	`/api/datasets/{id}/analyze`	Run and cache the full pipeline
GET	`/api/datasets/{id}/profile`	Cached profile
GET	`/api/datasets/{id}/metrics`	DQI, five components, informativeness
GET	`/api/datasets/{id}/informativeness`	Informativeness diagnostic
GET	`/api/datasets/{id}/issues`	Detected issues
GET	`/api/datasets/{id}/recommendations`	Recommendations
GET	`/api/datasets/{id}/model-performance`	Baseline results

Table 6.5 (continued)

Method	Endpoint	Purpose
GET	`/api/datasets/{id}/report?format=json|html`	Report export
POST	`/api/datasets/{id}/clean`	Auto-fix; new cleaned dataset
GET	`/api/datasets/{id}/download`	CSV download
GET	`/api/datasets/{id}/alternatives`	Similar datasets
GET	`/api/llm/status`	LLM configured and provider (never the key)
GET	`/api/discovery/providers`	Dataset sources and their state
GET	`/api/discovery/datasets`	Search catalogues
POST	`/api/discovery/import`	Import a catalogue dataset
GET	`/api/discovery/models`	Search models and repositories
GET	`/api/discovery/models/filters`	Allowed tasks, libraries, sorts

6.4.7 Discovery and the SSRF Guard

Listing 6.9: URL validation before every outbound download (`discovery/safe_http.py`)

```
def check_url(url: str) -> None:
    parts = urlsplit(url)
    host = (parts.hostname or "").lower().rstrip(".")
    if parts.scheme != "https":
        raise DownloadBlocked("Blocked a download: only HTTPS sources are allowed.")
    if parts.port not in (None, 443):
        raise DownloadBlocked("Blocked a download on a non-standard port.")
    try:
        ipaddress.ip_address(host)
        raise DownloadBlocked("Blocked a download from a raw IP address.")
    except ValueError:
        pass                                    # not an IP literal
    if not _host_allowed(host, parts.path or "/"):
```

```
14          raise DownloadBlocked(f"Blocked a download from a
    non-allowlisted host ({host}).")
15      for addr in _resolve_host(host):
16          if not ipaddress.ip_address(addr.split("%")[0]).is_global:
17              raise DownloadBlocked("Blocked a download that resolved to
    a "
18                                    "private or internal address.")
```

Searches across providers run concurrently, each in its own worker thread with a timeout; a failing or rate-limited provider returns an error status for itself only. The LLM client posts OpenAI-compatible requests to a fixed set of base URLs, retries up to twice on rate limits, server errors and network failures, falls back to a second provider, validates JSON output against a schema and raises a single error type whose message contains no key, URL or response body.

6.4.8 Frontend Implementation

The React application (Figure 6.1) uses plain state in `App.jsx` instead of a router: the top tabs switch between Analyze, Datasets and Models, and inside Analyze the six result pages receive the analysis object as a prop. The Datasets and Models tabs stay mounted after their first visit so that searches survive tab changes, and an imported dataset is passed to the upload page as if it had been uploaded. Each page is wrapped in an error boundary; network errors are shown as banners that distinguish "backend unreachable" from API errors.

FIGURE TO BE INSERTED

Frontend component tree: App (top tabs Analyze / Datasets / Models; header with Export Report, Export JSON, New Analysis) → Analyze: UploadPage (file drop, recent datasets, target and domain selectors) or AnalysisView (Nav → OverviewPage [DQIGauge, QualityCard x5, informativeness panel], IssuesPage [FilterButton by severity], ProfilePage [HistogramChart, CategoryDistributionChart, MissingValuesChart], ModelPage, RecommendationsPage [Download Corrected Dataset], SimilarDatasetsPage) ; Datasets: DatasetBrowserPage ; Models: ModelBrowserPage. Shared: ErrorBoundary, ErrorBanner, Spinner, SkeletonGrid, services/api.js.

Figure 6.1: Frontend component structure

6.5 Description of Usage

1. Start the backend and the frontend (Appendix G) and open `http://localhost:5173`.

2. On the *Analyze* tab, drop a CSV, Excel or JSON file, or reopen a dataset from "Recent datasets this session". Alternatively, find a dataset on the *Datasets* tab and click *Import into AutoDQ*.
3. Choose the target column (or leave auto-detect) and a domain preset, then click *Analyze Dataset*.
4. Review the *Overview* (DQI gauge, component cards, informativeness), *Issues* (filter by severity), *Dataset Profile* (per-column statistics and charts) and *Model Readiness* (cross-validated metrics with and without flagged columns).
5. On *Recommendations*, read the ranked actions and click *Download Corrected Dataset* to obtain a cleaned CSV with a before/after DQI comparison and the fix log.
6. Open *Similar Datasets* to see related public datasets with similarity scores, and import one to analyse it.
7. Use *Export Report* for a printable HTML report (save as PDF from the browser) or *Export JSON* for the full analysis.
8. Use the *Models* tab to search public models by task, library and popularity; links open the source page.

Chapter 7

System Testing

This chapter presents the testing approach, the test cases and their outcomes for the analysis engine, the API, the frontend workflow and the discovery features. Testing included both successful and failed cases; failed cases were either fixed and protected by an automated test, or are documented as open defects.

Testing Strategy

The following kinds of testing were applied:

- **Unit testing:** each analyzer, detector, formula and utility is tested in isolation, often against hand-computed values.
- **Ground-truth testing:** each synthetic dataset carries one known defect; the matching detector must find it and the clean dataset must not trigger false positives.
- **Directional validity testing:** degrading a dataset must lower its DQI.
- **Negative and edge-case testing:** empty data, malformed JSON, unsupported files, single-class targets, constant or entirely missing columns, undefined correlations.
- **Security testing:** SSRF attempts (non-allow-listed hosts, redirects to private addresses), oversized downloads, zip-slip archives, secret leakage in responses and logs.
- **Privacy testing:** no dataset cell value may appear in an outgoing LLM request.
- **Integration testing:** REST endpoints exercised end to end with FastAPI's test client; external services mocked.
- **Performance testing:** pipeline time measured on bundled datasets and on scaled copies of the churn dataset.
- **Acceptance testing:** a workflow walkthrough of every page and feature.

7.1 Unit Testing

Unit testing verifies individual functions in isolation from the rest of the workflow. Table 7.1 maps the most important units to the automated tests and test cases that verify them.

Table 7.1: Units under test and the tests that verify them

Unit	Behaviour verified (automated test)	Test cases
Ingestion	Empty and unsupported files raise errors; malformed JSON shapes handled (`test_empty_file_raises_ingestion_error`, `test_json_malformed_structure_handled`)	TC02–TC04
Completeness / Uniqueness	Exact formulas on hand-built frames (`test_completeness_formula`, `test_uniqueness_formula`)	TC10, TC11
Balance	1.0 for a 50/50 target; unavailable without a target	TC12, TC13
DQI engine	Weights sum to 1; proportional redistribution (`test_dqi_weights_sum_and_redistribution`)	TC13
Benjamini–Hochberg	Chance correlations suppressed in 40 random columns; genuine ones still found	TC23, TC24
Leakage detector	Leaked column flagged on dataset G	TC25
Multivariate probe	XOR leakage caught; quiet when already flagged or clean	TC26, TC27
Identifier heuristic	Continuous features not treated as IDs	TC28
Constant-column detector	Constant feature reported as an issue	TC29
Target auto-detection	Binary numeric target preferred over unrelated categorical	TC33
Baseline ML	Mean/std per metric; leakage excluded; only one of a redundant pair dropped	TC34–TC37
Informativeness	Separate from DQI; signal ranked above noise	TC39
Auto-fix	DQI improves; no-op on clean data; balance; untouched flagged columns; honest unfixable report; trimmed canonical values	TC42–TC47
SSRF guard	Non-allow-listed hosts, redirects and private IPs blocked	TC55, TC56
Download limits	Declared and streamed size limits; zip-slip; member caps	TC57, TC58
LLM client	Retries, fallback, JSON repair, no key in errors	TC62, TC63

Justification: the units chosen are those whose failure would directly produce a wrong score, a missed or false leakage finding, a misleading model metric, or a security exposure. Testing each on its own made it possible to reproduce every defect found during development and to prove its fix before the unit was combined with the rest of the pipeline.

7.2 Functional Testing

Functional testing verifies that each requirement (FR1–FR12) is met, including invalid inputs and edge cases. Actual results are taken from the automated tests and from re-running the system on the evaluation datasets.

7.2.1 Ingestion and Profiling

Table 7.2: Ingestion and profiling test cases

TC No.	Test case	Expected result	Actual result	Result
TC01	Upload a valid CSV (churn demo)	HTTP 200, dataset id, 10 typed columns	2120 × 10, no warnings	Pass
TC02	Upload an empty file	HTTP 400 with a clear message	"The uploaded file is empty."	Pass
TC03	Upload a `.txt` file	HTTP 400 naming supported types	"Unsupported file type '.txt'. Supported types: CSV, XLSX, JSON."	Pass
TC04	JSON as list, dict of columns, single object, nested records	Parsed with an explanatory warning	All shapes parsed; warnings added	Pass
TC05	Column of date strings	Typed datetime; pipeline completes	Typed datetime; validity checks parse dates	Pass
TC06	Empty DataFrame	Analysis completes with warnings	No exception; scores reported as not computable	Pass
TC07	Entirely missing column	Typed unknown; skipped in profile	Completed; column validity 0.5 (indeterminate)	Pass
TC08	Two-row dataset	Completes; baseline unavailable with reason	Completed; baseline reports that at least 30 rows are needed	Pass
TC09	Profile numeric and categorical columns	Statistics, histograms, top-10 categories	Produced for all typed columns	Pass

7.2.2 Quality Metrics and DQI

Table 7.3: Quality metric and DQI test cases

TC No.	Test case	Expected result	Actual result	Result
TC10	Completeness on a frame with known missing cells	$1 - \text{missing}/\text{cells}$ exactly	Matches hand calculation	Pass
TC11	Uniqueness on a frame with known duplicates	$1 - \text{duplicates}/\text{rows}$ exactly	Matches hand calculation	Pass
TC12	Balance of a 50/50 binary target	1.0	1.0	Pass
TC13	DQI with no target	Balance unavailable; other weights rescaled to sum to 1	Completeness weight 0.3125; sum 1.0	Pass
TC14	Clean dataset A	High DQI	100.0 (High)	Pass
TC15	Dataset B (missing values)	Lower DQI than A	$97.2 < 100.0$; Completeness 0.8875	Pass
TC16	Dataset C (imbalance)	Balance well below 1; ratio $\approx$1:9	Balance 0.466; ratio 1:9.1	Pass
TC17	Dataset H (combined problems)	Lower DQI than A	$89.3 < 100.0$	Pass
TC18	Low-quality demo	DQI in a low band	58.9 (Low)	Pass

7.2.3 Issue Detection and Recommendations

Table 7.4: Issue detection and recommendation test cases

TC No.	Test case	Expected result	Actual result	Result
TC19	Dataset B	Missing-value issues on both damaged columns	`feature_1` 15% (medium), `category` 30% (high)	Pass
TC20	Dataset D	Duplicate-row issue	10.71% duplicates (high)	Pass
TC21	Dataset C	Class-imbalance issue	Ratio 9.09 (high)	Pass
TC22	Dataset E	Outlier issue on `feature_1`	5.4% outliers (medium)	Pass
TC23	Dataset F	Redundant pair flagged	r = 0.9996 (high)	Pass

Table 7.4 (continued)

TC No.	Test case	Expected result	Actual result	Result
TC24	40 independent random columns, 60 rows	No redundancy flagged after BH	6 chance pairs at threshold 0.35 suppressed to 0	Pass
TC25	Dataset G	`target_echo` flagged as potential leakage	r = 0.9991 (high)	Pass
TC26	Target = a XOR b, both weakly correlated	Multivariate probe flags a and b	Held-out F1 = 1.0; cluster {a, b} flagged	Pass
TC27	Probe on clean data / with a univariate finding	No multivariate flag	No flag in either case	Pass
TC28	Continuous numeric features with near-unique values	Not treated as identifiers or leakage	Initially every continuous feature was dropped from the baseline and flagged as leakage; correct after the shared heuristic	Fail (fixed)
TC29	Dataset with three constant feature columns	Each reported as a visible issue	Initially only a hidden validity cap; now high-severity issues	Fail (fixed)
TC30	Recommendations for the churn demo	One per issue, critical → low	12 issues, 12 recommendations, severity-ordered	Pass
TC31	Redundancy scan when the target is numeric	Target not reported as "redundant" with a feature	Pair (`churn`, `retention_flag`) reported as redundant, with advice to remove one of them	Fail (open)
TC32	Leakage findings per column	One leakage issue per column	`target_echo` reported twice (name-based and correlation)	Fail (open)

7.2.4 Baseline Model and Informativeness

Table 7.5: Baseline model and informativeness test cases

TC No.	Test case	Expected result	Actual result	Result
TC33	No target given; 0/1 numeric target and an unrelated categorical column	0/1 column auto-detected	Initially the categorical column was chosen; now the binary target is chosen	Fail (fixed)
TC34	Baseline on dataset A	Stratified CV, mean ± std per metric	5 folds; Random Forest F1 0.904 ± 0.016	Pass
TC35	Baseline on dataset G	Second run without `target_echo`, lower score	F1 1.000 → 0.914	Pass
TC36	Baseline on dataset F	Only `feature_1_copy` excluded	Excluded `feature_1_copy`; F1 0.925 → 0.926	Pass
TC37	Baseline on dataset A	No second run	No exclusions; single run	Pass
TC38	Single-class target	Baseline unavailable with reason; no crash	"only one class exists" reported	Pass
TC39	One informative feature and one noise feature	Informative feature ranked first; not part of DQI	Ranked correctly; DQI components unchanged	Pass
TC40	Informativeness on churn demo	Leaked column dominates	`retention_flag` resolves 100% of target uncertainty	Pass
TC41	Identifier column in churn demo	`customer_id` excluded from features	`customer_id` used as a feature (2000 unique of 2120 rows, below the 98% rule because of duplicates)	Fail (open)

7.2.5 Auto-Fix and Report Export

Table 7.6: Auto-fix and report export test cases

TC No.	Test case	Expected result	Actual result	Result
TC42	Auto-fix on low-quality demo	DQI improves to a higher band	58.9 (Low) → 96.0 (High)	Pass

Table 7.6 (continued)

TC No.	Test case	Expected result	Actual result	Result
TC43	Auto-fix on clean dataset A	No changes	No-op; empty fix log	Pass
TC44	Auto-fix duplicates and imbalance	Duplicates removed; classes equal	485 duplicates removed; minority raised to majority count	Pass
TC45	Auto-fix with leaked and redundant columns	Those columns untouched	Values unchanged; still listed on Issues	Pass
TC46	Column whose values are all placeholders	Reported as unfixable, not invented	`notes`: "could NOT be filled"	Pass
TC47	Casing variants with trailing spaces	Canonical value has no stray whitespace	Initially "South " chosen; now trimmed form	Fail (fixed)
TC48	Export report	HTML and JSON returned	HTML 200 (text/html, ≈10.5 KB); JSON with dataset and analysis	Pass
TC49	Analyse with a non-existent target, then auto-fix	Fix uses the auto-detected target (Balance fixed)	Record stored the requested name; Balance step skipped (DQI 97.2 → 97.8 instead of 100.0)	Fail (open)

7.2.6 API and Frontend Workflow

Table 7.7: API and frontend workflow test cases

TC No.	Test case	Expected result	Actual result	Result
TC50	Request issues before analysing	HTTP 409	"Dataset has not been analyzed yet. Call /analyze first."	Pass
TC51	Unknown dataset id	HTTP 404	"Dataset not found."	Pass
TC52	Session history after several uploads	Each dataset listed with analysed flag and DQI	Listed correctly	Pass
TC53	Download corrected dataset	CSV attachment with `_cleaned` name	`demo_customer_churn_cleaned.csv`	Pass

Table 7.7 (continued)

TC No.	Test case	Expected result	Actual result	Result
TC54	Frontend served on a different port (Vite fallback to 5174)	Requests succeed	Initially failed with a CORS error when origins were restricted to 5173; default "*" and documented guidance added	Fail (fixed)

7.2.7 Discovery, Security and Privacy

Table 7.8: Discovery, security and privacy test cases

TC No.	Test case	Expected result	Actual result	Result
TC55	Download from a host not on the allow-list	Blocked	Blocked with a clean message	Pass
TC56	Redirect to a non-allow-listed host or a private IP	Blocked at the redirect hop	Blocked; auth header not forwarded	Pass
TC57	Response larger than the size limit	Aborted before or during streaming	Aborted on declared length and mid-stream	Pass
TC58	Zip with path traversal / too many members	Rejected	Rejected; largest valid file chosen otherwise	Pass
TC59	One provider fails or rate-limits	Other providers' results still returned	Per-provider error status; others returned	Pass
TC60	Inspect provider list, errors and logs	No keys or tokens anywhere	None found (Kaggle, OGD, HF, GitHub, LLM)	Pass
TC61	Invalid search parameters (task "xyz", 300-character query)	HTTP 422	422 "Unknown task 'xyz'."; 422 for long query	Pass
TC62	Similar Datasets with an LLM configured	No cell value in any outgoing prompt	Asserted over every captured request	Pass

Table 7.8 (continued)

TC No.	Test case	Expected result	Actual result	Result
TC63	LLM returns an unknown id / malformed output / no key	Unknown id dropped; fallback to keyword scoring	As expected; warning shown	Pass
TC64	Import World Bank indicator	Country × year table; aggregates dropped; row cap	As expected; cap enforced before paging	Pass

7.3 Integration Testing

7.3.1 Automated Test Suite

The automated suite exercises the engine directly and the REST API through FastAPI's test client, with HTTP transports and the LLM replaced by deterministic mocks. All 141 tests passed when re-run for this report (Table 7.9).

Table 7.9: Automated test suites

Suite	Tests	Scope
`test_core_engine.py`	48	Analyzers, DQI, detectors, BH, multivariate probe, baseline, informativeness, auto-fix, edge cases, ingestion
`test_discovery.py`	27	Hugging Face, OpenML, Kaggle; SSRF; size limits; archives; cache; import endpoint
`test_discovery_sources.py`	20	World Bank, OGD India, India Data Portal; catalogue sync; row caps; key redaction
`test_model_discovery.py`	17	Hugging Face models, GitHub, Vertex; filters; rate limits; tokens
`test_recommender.py`	29	Schema summary privacy, LLM client, query generation, scoring, re-ranking, endpoint
Total	**141**	**141 passed (85.7 s for the core suite, 11.9 s for the other four)**

7.3.2 End-to-End API Integration

A scripted run against the live application (FastAPI test client) performed the complete workflow on the churn dataset: upload (HTTP 200), analyse (DQI 97.2, 12 issues), fetch every result endpoint, export HTML and JSON reports, run auto-fix, download the cleaned CSV, list the session history, query the provider list and model filters, and send invalid parameters. All responses matched the expected status codes and shapes; the OpenAPI schema lists 22 paths (the 20 API endpoints plus `/` and `/health`).

7.3.3 Performance Testing

Table 7.10 reports wall-clock times for the complete pipeline (ingestion plus analysis, including two cross-validated models, a possible second run and the leakage probe) on a 2-vCPU machine. Ingestion takes under 10 ms for every bundled file. Scaling the churn dataset tenfold increases the time only about 2.5-fold, because the fixed costs of model fitting dominate at these sizes.

Table 7.10: Pipeline execution time (2 vCPU)

Dataset	Rows	File size	Total time
A_clean.csv	1,000	40 KB	2.6 s
H_combined_problems.csv	621	46 KB	4.9 s
low_quality_demo.csv	850	26 KB	5.1 s
demo_customer_churn.csv	2,120	125 KB	5.8 s
churn × 5	10,600	—	10.0 s
churn × 10	21,200	—	14.7 s

7.4 Acceptance Testing

7.4.1 User Acceptance Testing

Acceptance was carried out as a walkthrough of the complete workflow, following the demonstration script: upload the churn dataset, auto-detect the target, analyse, review all six pages, download the corrected dataset, export the report, and use the Datasets and Models tabs. Table 7.11 lists the workflows verified; Table 7.12 is reserved for feedback from reviewers and users.

Table 7.11: Workflow acceptance walkthrough

Area	**Workflow verified**	**Outcome**
Upload and setup	File upload, warnings, recent datasets, target auto-detection, domain preset	Accepted
Overview	DQI gauge, band, five component cards with weights and contributions, informativeness panel	Accepted
Issues	Severity filter, column names, measured values, explanations	Accepted; open defects TC31, TC32
Dataset Profile	Statistics, histograms, category charts, missing-value chart	Accepted
Model Readiness	Mean $\pm$ std per metric; all features versus flagged columns removed; readiness label	Accepted; open defect TC41
Recommendations and auto-fix	Ranked actions; corrected dataset; before/after DQI; fix log; CSV download	Accepted; open defect TC49
Reports	HTML report printed to PDF; JSON export	Accepted
Discovery	Dataset search and import; model search with filters; similar datasets with and without an LLM key	[to be confirmed with live network access]

Table 7.12: User feedback summary (ratings out of 5)

Feature	Rating (5)	Feedback
Ease of uploading and analysing a dataset	[–]	[to be collected]
Clarity of the DQI and its breakdown	[–]	[to be collected]
Usefulness of issues and recommendations	[–]	[to be collected]
Leakage-aware model readiness	[–]	[to be collected]
Corrected dataset (auto-fix)	[–]	[to be collected]
Dataset and model discovery	[–]	[to be collected]

Overall, the system passed the critical test cases. Five defects found during development were fixed and are covered by automated tests; four defects found during the final review remain open and are listed below.

Justification: acceptance testing followed the complete workflow because the purpose of the project is that a user without programming effort can go from a raw file to an evidence-based verdict and a corrected dataset; the walkthrough also exposed defects (TC31, TC32, TC41, TC49) that the unit tests did not cover.

7.5 Defect Summary

Table 7.13: Summary of failed test cases and their resolution

TC No.	Issue observed	Status	Action taken / planned
TC28	Cardinality-only ID rule treated continuous features as identifiers: real features dropped from the baseline and flagged as leakage	Fixed	Shared `is_id_like_column` requiring integer values and an ID-like name for numeric columns; used by all callers
TC29	Constant columns visible only as a hidden validity cap	Fixed	New high-severity `constant_column` issue and recommendation

Table 7.13 (continued)

TC No.	Issue observed	Status	Action taken / planned
TC33	0/1 numeric targets invisible to auto-detection	Fixed	Binary numeric columns added as candidates
TC47	Canonical category value kept trailing whitespace	Fixed	Most frequent *trimmed* variant chosen
TC54	CORS failure when Vite moved to another port	Fixed	Default "*" for local use; documented guidance in `.env.example`
—	Validation harness injected outliers into a 0/1 leakage column, hiding the leak and inflating scores	Fixed	Outliers injected only into columns with more than two distinct values
TC31	Target column included in the redundancy scan	Open	Exclude the target from `detect_redundancy`; leakage checks already cover it
TC32	Same column reported twice for leakage	Open	Merge leakage findings per column, keeping the highest severity and all reasons
TC41	`customer_id` used as a model feature when duplicates lower its uniqueness	Open	Evaluate uniqueness on deduplicated rows, or treat ID-named integer columns more strongly
TC49	Stored target is the requested (invalid) name instead of the resolved one	Open	Store the target actually used by the pipeline

Chapter 8

Results and Discussions

AutoDQ was implemented end to end and evaluated on datasets with known defects, through 141 automated tests, and through a degradation study. The results show that the framework scores data quality transparently, detects each injected defect, exposes target leakage with model-based evidence, and repairs the five quality dimensions automatically. All figures in this chapter were reproduced from the submitted code for this report.

8.1 Functional Achievements

Table 8.1: Summary of achieved functional objectives

Objective	Implementation	Status
Robust ingestion	CSV, Excel and four JSON shapes; recoverable problems reported as warnings	Achieved
Explainable quality index	Five analyzers; weighted DQI with four presets, redistribution and per-component contributions	Achieved
Concrete issue detection	Nine detectors with severities; BH correction; multivariate leakage probe	Achieved (2 open defects)
Actionable recommendations	One message per issue, severity-ordered	Achieved
Learnability with evidence	Stratified CV, two models per task, second run without flagged columns, readiness label	Achieved (1 open defect)
Hygiene versus signal	Mutual-information informativeness, kept outside the DQI	Achieved
Automated correction	Auto-fix for the five dimensions; new dataset; before/after DQI; fix log; CSV	Achieved (1 open defect)

Table 8.1 (continued)

Objective	Implementation	Status
Reporting	Printable HTML report; JSON export; session history	Achieved
Dataset and model discovery	Six dataset sources, three model sources, import into the analysis flow	Achieved
Similar dataset recommendation	Structural similarity with optional, privacy-preserving LLM re-ranking	Achieved
Security and privacy	SSRF guard, size and row caps, secret redaction, no cell values to the LLM	Achieved
Validation	141 automated tests; 11 evaluation datasets; degradation harness	Achieved

8.2 Quality Scores Across the Evaluation Datasets

Table 8.2 reports the five component scores, the DQI and the number of issues for every evaluation dataset. Each injected defect lowers exactly the component designed to measure it: missing values lower Completeness (B), duplicates lower Uniqueness (D), and imbalance lowers Balance (C). The heavily damaged dataset falls into the Low band.

Table 8.2: DQI and component scores on the evaluation datasets (General preset)

Dataset	C	S	V	U	B	DQI	Band	Issues
A_clean	1.000	1.000	1.000	1.000	1.000	100.0	High	2
B_missing_values	0.888	1.000	1.000	1.000	1.000	97.2	High	4
C_class_imbalance	1.000	1.000	1.000	1.000	0.466	89.3	High	3
D_duplicates	1.000	1.000	1.000	0.893	1.000	98.4	High	3
E_outliers	1.000	1.000	1.000	1.000	1.000	100.0	High	2
F_redundant_features	1.000	1.000	1.000	1.000	1.000	100.0	High	4
G_leakage	1.000	1.000	1.000	1.000	1.000	100.0	High	5
H_combined_problems	0.967	1.000	1.000	0.926	0.563	89.3	High	12
demo_customer_churn	0.979	1.000	1.000	0.994	0.890	97.2	High	12
low_quality_demo	0.509	0.875	0.900	0.533	0.135	58.9	Low	12
high_readiness_demo	1.000	1.000	1.000	1.000	1.000	100.0	High	3

C = Completeness, S = Consistency, V = Validity, U = Uniqueness, B = Balance.

Outliers (E), redundancy (F) and leakage (G) leave the DQI at 100 by design: they are not data-hygiene defects in the sense of the five dimensions, so they are reported as issues and tested through the baseline instead of being folded into the score. This separation is what allows the DQI to stay interpretable, but it also means that *a high DQI alone is not evidence that a dataset is safe to model*; the Issues and Model Readiness pages must be read together with it.

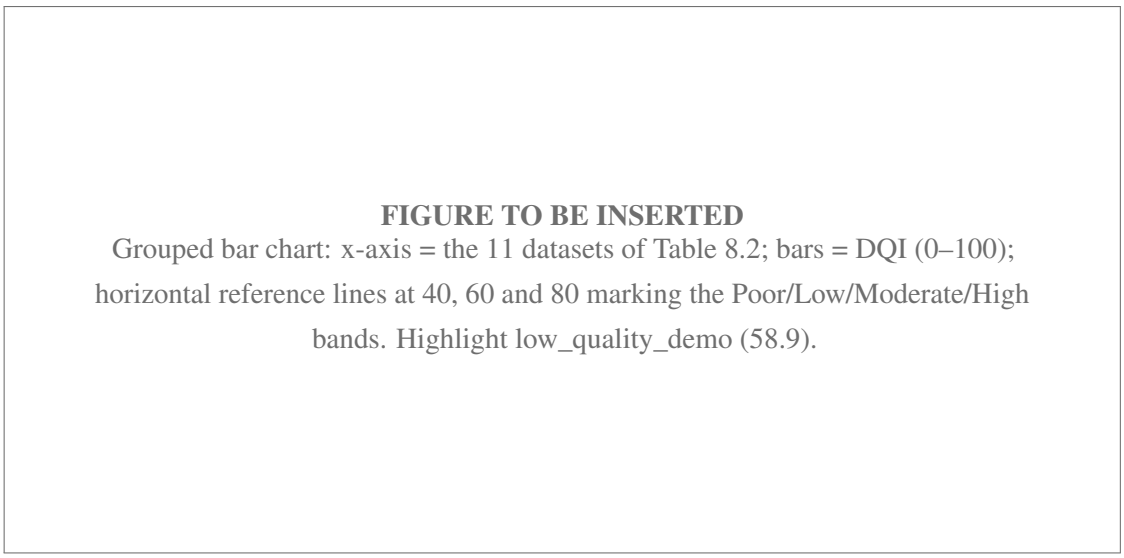


Figure 8.1: DQI of each evaluation dataset with interpretation bands

8.3 Leakage-Aware Baseline Results

Table 8.3 compares the best cross-validated macro-F1 with all usable features and with the flagged columns removed. Whenever a leaked column is present, both models reach a perfect score; removing it reveals the realistic performance. Removing only a redundant copy (F) leaves performance unchanged, which confirms that the exclusion rule keeps the signal of a correlated pair.

Table 8.3: Best macro-F1 (5-fold CV) with all features and with flagged columns removed

Dataset	Columns excluded	All features	Flagged removed	Readiness
G_leakage	`target_echo`	1.000	0.914	High
H_combined_problems	`feature_1_copy`, `target_echo`	1.000	0.832	Moderate
demo_customer_churn	`estimated_annual_value`, `retention_flag`	1.000	0.639	Low
F_redundant_features	`feature_1_copy`	0.925	0.926	High
A_clean	—	0.904	—	High
C_class_imbalance	—	0.883	—	High

Table 8.4: Per-model results on dataset H (mean ± std over 5 folds)

Feature set	Model	Accuracy	Precision	Recall	F1
All features	Logistic Regression	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
All features	Random Forest	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
Flagged removed	Logistic Regression	0.898 ± 0.017	0.837 ± 0.063	0.652 ± 0.052	0.695 ± 0.064
Flagged removed	Random Forest	0.928 ± 0.017	0.862 ± 0.044	0.809 ± 0.023	0.832 ± 0.030

Dataset H also illustrates why macro-averaged metrics are used: with a 1:6.6 class ratio, Logistic Regression reaches 0.898 accuracy but only 0.652 macro recall, because it under-predicts the minority class. On the churn demo the honest F1 of 0.639 earns a “Low” readiness label, so the demonstration shows a dataset that looks excellent until its leaked column is removed.

FIGURE TO BE INSERTED
Horizontal grouped bar chart from Table 8.3: for G, H, churn and F, a grey bar "All features" and a coloured bar "Flagged columns removed" (macro-F1, axis 0–1); dashed reference line at F1 = 0.85 (High readiness threshold). Values: G 1.000/0.914, H 1.000/0.832, churn 1.000/0.639, F 0.925/0.926.

Figure 8.2: Effect of removing flagged columns on cross-validated macro-F1

8.4 Auto-Fix Results

Table 8.5: Auto-fix results (re-analysed corrected datasets)

Dataset	DQI before	DQI after	Rows	Main corrections
low_quality_demo	58.9 (Low)	96.0 (High)	850 → 710	199 placeholders cleared; 10 casing variants merged; 3,527 cells imputed; 485 duplicates dropped; 345 minority rows added; `notes` unfixable
H_combined_problems	89.3	100.0	621 → 1000	122 cells imputed; 46 duplicates dropped; 425 minority rows added
demo_customer_churn	97.2	100.0	2120 → 2926	450 cells imputed; 12 duplicates dropped; 818 minority rows added

On the low-quality dataset the component scores moved from Completeness 0.51 → 0.90, Consistency 0.88 → 1.00, Uniqueness 0.53 → 1.00 and Balance 0.13 → 1.00. Validity rose only to 0.93 because the `notes` column contained nothing but placeholders; after they were converted to missing values there was nothing real to impute from, and the engine reported this instead of inventing values. Columns flagged as redundant (`income_copy`, `age_copy`) were left untouched for the user to decide.

Note that the corrected dataset is re-scored on the same five dimensions that the engine repairs, so a high post-fix DQI is expected by construction. Model performance after auto-fix should be interpreted with care: the balance step adds synthetic minority rows before cross-

validation, so synthetic rows derived from the same class can appear in both training and test folds and make post-fix metrics optimistic.

8.5 Validation Harness Results

The harness degrades a dataset at five intensities and measures the DQI and the baseline's primary metric at each level (Table 8.6).

Table 8.6: DQI and macro-F1 under controlled degradation

Degradation intensity	A_clean DQI	A_clean F1	Churn DQI	Churn F1
0.00	100.0	0.904	97.2	0.639
0.25	97.2	0.858	94.3	0.625
0.50	94.2	0.813	92.2	0.638
0.75	91.5	0.790	90.0	0.627
1.00	87.9	0.731	88.2	0.620
Pearson *r* (DQI, F1)	**0.996**		**0.683**	

On A_clean, DQI and F1 fall together almost linearly. On the churn dataset the correlation is positive but weaker, and F1 changes by only about 0.02 across all levels: the remaining signal is limited (F1 $\approx$ 0.64 even without degradation), so the added noise has little left to destroy. Both datasets were built by the project team, so these results demonstrate the mechanism rather than establishing a general relationship; an independent study on real datasets is the main future work.

FIGURE TO BE INSERTED

Dual-axis line chart: x-axis = degradation intensity (0, 0.25, 0.5, 0.75, 1.0); left axis = DQI (85–100); right axis = macro-F1 (0.6–0.95). Two panels or four lines: A_clean DQI and F1 (r = 0.996), churn DQI and F1 (r = 0.683), using values from Table 8.6.

Figure 8.3: DQI and baseline F1 under increasing degradation

8.6 Graphical Analysis of Test Results

Of the 64 documented test cases, 55 passed on first execution, 5 failed and were fixed, and 4 remain open (Table 6.1). All 141 automated tests pass.

FIGURE TO BE INSERTED
Stacked horizontal bar chart per test group (Ingestion 9; Metrics/DQI 9; Issues 14; Baseline 9; Auto-fix/Export 8; API/Frontend 5; Discovery/Security 10): segments “Passed first time” (9, 9, 10, 7, 6, 4, 10), “Failed then fixed” (0, 0, 2, 1, 1, 1, 0), “Open” (0, 0, 2, 1, 1, 0, 0).

Figure 8.4: Test case outcomes by group

8.7 Code Base Composition

FIGURE TO BE INSERTED
Horizontal bar chart of lines of code per component from Table 6.3: services 2,754; discovery 2,180; tests 2,381; frontend 1,942; docs 1,610; analyzers 670; utilities/DQI/recommendations/schemas 461; API routes 437; generator and harness 310.

Figure 8.5: Lines of code per component

8.8 Justification of Requirements

Tables 8.7 and 8.8 justify each requirement of Chapter 3 with the evidence obtained in testing.

Table 8.7: Justification of functional requirements

FR	Requirement	Evidence of fulfilment
FR1	Accept CSV/Excel/JSON; reject invalid files; warn on recoverable problems	TC01–TC04; edge cases TC06–TC08 complete without errors
FR2	Infer types and profile columns	TC05, TC07, TC09
FR3	Target and domain selection; auto-detection	TC33 (fixed); TC49 records an open defect in how the resolved target is stored
FR4	Five metrics and DQI with breakdown	TC10–TC18; Table 8.2
FR5	Nine issue types with severity and explanation	TC19–TC29; TC31, TC32 open
FR6	One recommendation per issue, ordered	TC30
FR7	Cross-validated baseline with and without flagged columns	TC34–TC38; Table 8.3; TC41 open
FR8	Informativeness diagnostic	TC39, TC40
FR9	Corrected dataset with before/after comparison	TC42–TC47, TC53; Table 8.5
FR10	HTML/JSON export; session history	TC48, TC52
FR11	Dataset search/import; model search	TC59, TC61, TC64; discovery suites (64 tests)
FR12	Similar dataset recommendation	TC62, TC63; recommender suite (29 tests)

Table 8.8: Justification of non-functional requirements

Requirement	Evidence of fulfilment
Explainability	Every component's score, weight, effective weight and contribution returned by the API; every issue carries its measured value and explanation; formulas documented in Chapter 5.
Statistical soundness	Preprocessing refitted per fold; stratified CV; BH correction (TC24); leakage worded as "potential".

Table 8.8 (continued)

Requirement	Evidence of fulfilment
Robustness	Ten automated edge-case tests (empty data, missing target, all-missing, single-class, constant and tiny datasets, and others) complete without exceptions; TC04–TC08 and TC38.
Security	SSRF, size, redirect and archive tests pass (TC55–TC58); secrets never returned or logged (TC60).
Privacy	No cell values in LLM prompts (TC62); optional hashing of column names.
Performance	2.6–5.8 s per bundled dataset and 14.7 s for 21,200 rows on 2 vCPUs (Table 7.10); result pages read the cache.
Usability	Complete workflow without code accepted in the walkthrough (Table 7.11).
Maintainability and testability	Modular engine tested without the web server; shared heuristics in single modules; 141 automated tests.
Portability	Configuration via environment variables with defaults; developed on Windows, re-run on Linux.

8.9 Comparison with Existing Systems

Table 8.9 compares AutoDQ with the categories of existing tools reviewed in Chapter 2, based on their published descriptions. A tick means the capability is provided, "partial" means it is provided in a limited form or requires user-written rules, and a cross means it is not a goal of that category.

Table 8.9: Feature comparison with existing categories of tools

Capability	**Profiling reports**	**Validation frameworks** [9, 10]	**ML quality toolkits** [11, 12]	**AutoDQ**
Column statistics and distributions	✓	partial	partial	✓
Rule checks without writing rules	partial	✗	partial	✓
Single explainable quality index	✗	✗	partial	✓
Severity-ranked issues with explanations	partial	partial	✓	✓

Table 8.9 (continued)

Capability	Profiling	Validation	ML toolkits	AutoDQ
Multiple-comparisons correction	✗	✗	✗	✓
Univariate leakage detection	✗	✗	partial	✓
Multivariate leakage detection	✗	✗	✗	✓
Cross-validated model evidence	✗	✗	✗	✓
Hygiene and signal reported separately	✗	✗	partial	✓
Automated correction with re-scoring	✗	✗	partial	✓
Code-free web interface	partial	✗	partial	✓
Large-scale / streaming data	✗	✓	partial	✗
Schema drift across batches	✗	✓	✗	✗
Dataset and model discovery	✗	✗	✗	✓

AutoDQ does not replace production validation frameworks: it does not process data at Spark scale or track drift across pipeline runs. Its contribution is the pre-modelling assessment of a single dataset, with model-based evidence and correction, in a form that requires no programming.

8.10 Visual Results of UI Implementation

The following screenshots are to be captured from the running application using `demo_customer_churn.csv` (target `churn`, General preset) unless stated otherwise.

8.10.1 Upload and Overview

FIGURE TO BE INSERTED
Screenshot: Analyze tab – Upload page with a selected file, column list and warnings, target selector (auto-detect / churn), domain preset selector, "Recent datasets this session" panel and the Analyze Dataset button.

Figure 8.6: Upload page with target and domain selection

FIGURE TO BE INSERTED
Screenshot: Overview page – DQI gauge (97.2, High), five component cards with score, weight and contribution, warnings, and the separate experimental Informativeness panel showing retention_flag at 100%.

Figure 8.7: Overview page with DQI gauge and component breakdown

8.10.2 Issues and Dataset Profile

FIGURE TO BE INSERTED
Screenshot: Issues page – severity filter buttons (critical/high/medium/low) and issue cards for the churn dataset, including redundant features (monthly_charges / estimated_annual_value) and potential leakage (retention_flag).

Figure 8.8: Issues page with severity filters

FIGURE TO BE INSERTED
Screenshot: Dataset Profile page – missing-values bar chart, a numeric column histogram (e.g. tenure_months) with statistics, and a category distribution chart (contract_type).

Figure 8.9: Dataset profile with histograms and category charts

8.10.3 Model Readiness

FIGURE TO BE INSERTED
Screenshot: Model Readiness page – task type, folds, readiness label (Low), table of Logistic Regression and Random Forest metrics as mean ± std for "All features" and "Excluding flagged leakage/redundant features", and the excluded columns.

Figure 8.10: Model readiness with the leakage-aware comparison

8.10.4 Recommendations and Auto-Fix

FIGURE TO BE INSERTED
Screenshot: Recommendations page – severity-ordered recommendations and the Download Corrected Dataset result panel for low_quality_demo.csv showing DQI 58.9 → 96.0, rows 850 → 710 and the fix log.

Figure 8.11: Recommendations and auto-fix before/after comparison

8.10.5 Similar Datasets and Discovery

FIGURE TO BE INSERTED
Screenshot: Similar Datasets page – ranked cards with similarity %, score breakdown (column overlap, type, task, size, LLM), metadata coverage, reason, View source and Import buttons; LLM status banner.

Figure 8.12: Similar datasets page

FIGURE TO BE INSERTED
Screenshot: Datasets tab – search box, provider filter and per-provider status chips (Hugging Face, OpenML, Kaggle, World Bank, OGD India, India Data Portal), result cards with Import into AutoDQ.

Figure 8.13: Dataset Browser

FIGURE TO BE INSERTED
Screenshot: Models tab – task, library and sort filters, "Show all task types" toggle, model and repository cards with downloads/stars and View source links.

Figure 8.14: Model Browser

8.10.6 Report Export and API Documentation

FIGURE TO BE INSERTED
Screenshot: exported HTML report opened in the browser (DQI table, issues, recommendations, baseline, informativeness) – first page.

Figure 8.15: Exported HTML quality report

FIGURE TO BE INSERTED
Screenshot: FastAPI interactive documentation at http://localhost:8000/docs
listing the datasets, discovery and models endpoint groups.

Figure 8.16: Interactive API documentation

8.11 Discussion

The results indicate that AutoDQ meets its central objective: a user can go from a raw file to an explainable quality verdict, a ranked list of concrete problems, model-based evidence and a corrected dataset in seconds and without code. Each synthetic defect is detected by the detector designed for it, the clean dataset does not produce false positives for missing values or duplicates, and degrading data lowers the DQI.

The most persuasive result is the leakage-aware baseline. A model that scores a perfect 1.00 looks like success; the second run, which drops the columns AutoDQ flagged, turns a warning into a measurement (0.91 on G, 0.83 on H, 0.64 on the churn demo). The informativeness diagnostic reached the same conclusion by an independent method, giving `retention_flag` 100% of the target's information. The Benjamini–Hochberg correction and the multivariate probe extend detection in two directions that fixed univariate thresholds cannot reach.

The project also shows the value of validating the validator. Testing on real and synthetic data uncovered a heuristic that silently removed continuous features, a target detector blind to 0/1 targets, and a harness whose own noise hid the leakage it was meant to measure. The final review of this report surfaced four further defects, which are documented rather than hidden.

Limitations Identified

- The relationship between DQI and model performance has been demonstrated only on datasets built by the team (correlations 0.996 and 0.683); it has not yet been validated on independently sourced data, and the weights and bands are uncalibrated heuristics.
- The DQI measures hygiene, not signal or leakage; outliers, redundancy and leakage leave it unchanged and must be read from the Issues and Model Readiness pages.

- The multivariate probe is a single model fit, not an exhaustive search; group purity has no multiple-comparisons correction.
- The balance correction is an approximation of synthetic oversampling that does not preserve joint feature relationships, and post-fix model metrics can be optimistic.
- Near-duplicates that differ in a noisy derived column are not detected (Section 6.2).
- Four open defects (TC31, TC32, TC41, TC49) remain.
- Data is held in memory only; there are no user accounts, and the system is not designed for very large datasets.

Conclusion of the Results

The outcomes demonstrate that an integrated, explainable and data-centric framework can make dataset assessment fast, consistent and evidence-based. Transparent scoring, statistically controlled detection, leakage-aware model evaluation and automated correction together confirm that the design is technically sound and ready to be validated on a wider range of real datasets.

Conclusion and Future Enhancements

Conclusion

AutoDQ turns the assessment of a tabular dataset from a manual, inconsistent activity into an automated, explainable and evidence-based one. From a single uploaded or imported file, the framework infers the schema, profiles every column, scores five quality dimensions and combines them into a transparent Dataset Quality Index, detects nine kinds of concrete problems, recommends a specific action for each, and measures how well real models learn from the data. A corrected dataset can be produced and re-scored with one click.

Three design decisions distinguish the work. First, the quality index is deliberately simple and traceable, and it is kept separate from both predictive signal (measured by mutual information) and leakage (measured by detectors and model evidence), so that a low score has one meaning. Second, detection is statistically disciplined: correlation findings must survive Benjamini–Hochberg correction, and a model-based probe catches leakage that no single-column check can see. Third, every warning about leakage is backed by evidence on the user's own data: on the synthetic leakage dataset, removing the flagged column reduced macro-F1 from 1.00 to 0.91, and on the churn demonstration from 1.00 to 0.64.

The system is supported by 141 passing automated tests, eleven datasets with known defects, and a degradation harness in which DQI and model performance moved together (correlations of 0.996 and 0.683). Secure dataset discovery, a model browser and privacy-preserving similar-dataset recommendations extend the framework from diagnosing data to helping users find better data. The project also documents its own limits honestly: the DQI weights are uncalibrated heuristics, the relationship to performance has not yet been tested on independent datasets, and four defects found during the final review remain open.

Future Enhancements

- **Independent validation study:** run the validation harness on 5–10 independently sourced public datasets (for example from UCI, OpenML and Kaggle) and use the results to calibrate the DQI weights and interpretation bands.
- **Close the open defects:** exclude the target from the redundancy scan, merge duplicate leakage findings, detect identifiers after de-duplication, and store the resolved target (TC31, TC32, TC41, TC49).
- **Stronger balancing:** replace the class-conditional resampling with SMOTE or SMOTE-

NC, and apply resampling inside each cross-validation fold so that post-fix metrics stay unbiased.

- **Informativeness as a dimension:** study whether mutual-information informativeness should become a weighted sixth DQI component, with re-calibrated weights.
- **More detectors:** Isolation Forest for multivariate outliers, a permutation test for group purity, near-duplicate detection that tolerates noisy derived columns, and label-noise detection using confident learning.
- **Persistence and collaboration:** store datasets and analyses in a database, add user accounts, and track DQI across versions of the same dataset.
- **Scale:** chunked or out-of-core processing for datasets larger than memory.
- **New data types:** profiling tracks for text and image datasets that produce the same tabular feature representation used by the existing analyzers.
- **Model suggestions and AI explanations:** recommend model families suited to the analysed dataset, and generate plain-language explanations of the DQI breakdown without sending data values to external services.
- **Learned recommendation ranking:** rank fixes by the improvement they historically produced in F1 or R^2.

References

[1] A. Jain, H. Patel, L. Nagalapatti, N. Gupta, S. Mehta, S. Guttula, S. Mujumdar, S. Afzal, R. Sharma Mittal, and V. Munigala, "Overview and importance of data quality for machine learning tasks," in *Proc. 26th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining (KDD)*, 2020.

[2] S. Mohammed, L. Budach, M. Feuerpfeil, N. Ihde, A. Nathansen, N. Noack, H. Patzlaff, F. Naumann, and H. Harmouch, "The effects of data quality on machine learning performance on tabular data," *Information Systems*, vol. 132, Art. no. 102549, 2025, doi: 10.1016/j.is.2025.102549.

[3] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2015, pp. 2503–2511.

[4] N. Sambasivan, S. Kapania, H. Highfill, D. Akrong, P. Paritosh, and L. M. Aroyo, " 'Everyone wants to do the model work, not the data work': Data cascades in high-stakes AI," in *Proc. CHI Conf. on Human Factors in Computing Systems (CHI '21)*, 2021, doi: 10.1145/3411764.3445518.

[5] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, Art. no. 100804, 2023, doi: 10.1016/j.patter.2023.100804.

[6] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive label errors in test sets destabilize machine learning benchmarks," in *Proc. 35th Conf. on Neural Information Processing Systems (NeurIPS), Track on Datasets and Benchmarks*, 2021.

[7] D. Zha, Z. P. Bhat, K.-H. Lai, F. Yang, Z. Jiang, S. Zhong, and X. Hu, "Data-centric artificial intelligence: A survey," *ACM Computing Surveys*, 2025, doi: 10.1145/3711118 (preprint arXiv:2303.10158, 2023).

[8] S. E. Whang, Y. Roh, H. Song, and J.-G. Lee, "Data collection and quality challenges in deep learning: A data-centric AI perspective," *The VLDB Journal*, vol. 32, 2023.

[9] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[10] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Conf. on Machine Learning and Systems (SysML/MLSys)*, 2019.

[11] N. Hynes, D. Sculley, and M. Terry, "The Data Linter: Lightweight, automated sanity checking for ML data sets," in *NIPS Workshop on Machine Learning Systems*, 2017.

[12] N. Gupta *et al.*, "Data Quality Toolkit: Automatic assessment of data quality and remediation for machine learning datasets," arXiv:2108.05935, 2021.

[13] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. on Knowledge Discovery from Data*, vol. 6, no. 4, Art. 15, 2012.

[14] ISO/IEC 25012:2008, *Software engineering – Software product Quality Requirements and Evaluation (SQuaRE) – Data quality model*. Geneva, Switzerland: International Organization for Standardization, 2008.

[15] R. Y. Wang and D. M. Strong, "Beyond accuracy: What data quality means to data consumers," *J. of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996.

[16] L. L. Pipino, Y. W. Lee, and R. Y. Wang, "Data quality assessment," *Communications of the ACM*, vol. 45, no. 4, pp. 211–218, 2002.

[17] C. Batini, C. Cappiello, C. Francalanci, and A. Maurino, "Methodologies for data quality assessment and improvement," *ACM Computing Surveys*, vol. 41, no. 3, Art. 16, 2009.

[18] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[19] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[20] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. of the Royal Statistical Society: Series B*, vol. 57, no. 1, pp. 289–300, 1995.

[21] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 1995, pp. 1137–1145.

[22] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[23] A. Kraskov, H. Stögbauer, and P. Grassberger, "Estimating mutual information," *Physical Review E*, vol. 69, no. 6, Art. 066138, 2004.

[24] B. C. Ross, "Mutual information between discrete and continuous data sets," *PLoS ONE*, vol. 9, no. 2, Art. e87357, 2014.

[25] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[26] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[27] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[28] L. R. Dice, "Measures of the amount of ecologic association between species," *Ecology*, vol. 26, no. 3, pp. 297–302, 1945.

[29] OWASP Foundation, "OWASP API Security Top 10 – 2023," 2023. [Online]. Available: `https://owasp.org/API-Security/editions/2023/en/0x11-t10/`

[30] OWASP Cheat Sheet Series, "Server-Side Request Forgery prevention cheat sheet." [Online]. Available: `https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html`

[31] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.

[32] Q. Lhoest *et al.*, "Datasets: A community library for natural language processing," in *Proc. 2021 Conf. on Empirical Methods in Natural Language Processing: System Demonstrations*, 2021, pp. 175–184.

[33] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *ACM SIGKDD Explorations*, vol. 15, no. 2, pp. 49–60, 2013.

[34] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.

[35] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[36] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[37] S. Ramírez, "FastAPI." [Online]. Available: `https://fastapi.tiangolo.com`

[38] Meta Open Source, "React – The library for web and native user interfaces." [Online]. Available: `https://react.dev`

APPENDIX A

Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
BH	Benjamini–Hochberg (false discovery rate procedure)
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
CV	Cross-Validation
DCAI	Data-Centric Artificial Intelligence
DFD	Data Flow Diagram
DNS	Domain Name System
DQI	Dataset Quality Index
DTO	Data Transfer Object
F1	Harmonic mean of precision and recall
FDR	False Discovery Rate
FR	Functional Requirement
HTML	HyperText Markup Language
HTTP(S)	HyperText Transfer Protocol (Secure)
IQR	Inter-Quartile Range
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
JSON	JavaScript Object Notation
LLM	Large Language Model
LOC	Lines of Code
MAE	Mean Absolute Error
MI	Mutual Information
ML	Machine Learning
NFR	Non-Functional Requirement
OGD	Open Government Data (Platform India)
OWASP	Open Worldwide Application Security Project

Abbreviation	Expansion
REST	Representational State Transfer
RMSE	Root Mean Squared Error
R^2	Coefficient of determination
SDG	Sustainable Development Goal
SMOTE	Synthetic Minority Over-sampling Technique
SPA	Single-Page Application
SRS	System Requirement Specification
SSRF	Server-Side Request Forgery
TC	Test Case
TFDV	TensorFlow Data Validation
TLS	Transport Layer Security
TTL	Time To Live
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
USN	University Seat Number
WP	Washington Accord Problem-solving attribute
XLSX	Office Open XML spreadsheet format
XOR	Exclusive OR

APPENDIX B

Complex Engineering Problem

Mapping of Complex Engineering Problems with Washington Accord WPs (WP1–WP7)

The following table maps the major project against the Washington Accord knowledge and problem-solving attributes (WP1–WP7). A tick (✓) in the Applicable column indicates that the competency is demonstrated in this project; a dash (–) indicates that it is not claimed. The evidence for each tick is summarised after the tables.

WP Code	Description	Competencies	Applicable
WP1	In-Depth Engineering Knowledge	Uses advanced algorithms	✓
		Works with complex data structures	✓
		Applies specialised domain knowledge	✓
		Uses professional-grade tools and platforms	✓
		Combines knowledge from multiple CSE fields	✓
		Performs algorithm optimization	–
		Refers to research papers or standards	✓
WP2	Wide-Ranging or Conflicting Technical & Non-Technical Issues	Balances security with performance	✓
		Evaluates resource limitations	✓
		Considers user experience needs	✓
		Follows privacy and security rules	✓
		Chooses scalable, maintainable solutions	✓
		Identifies and manages risks	✓
		Considers ethical impacts	✓
WP3	Abstract Thinking & Originality	Designs new algorithms or models	✓
		Creates original system designs	✓
		Introduces innovations in ML models	–
		Applies strong abstract reasoning	✓
		Develops optimization improvements	–

WP Code	Description	Competencies	Applicable
		Experiments with new data representations	✓
		Adds ideas beyond standard tutorials	✓
WP4	Design and Development of Solutions	Solves real-world problems without ready-made code	✓
		Modifies existing tools/frameworks in advanced ways	✓
		Uses cutting-edge technologies	✓
		Implements algorithms/protocols from scratch	✓
		Handles incomplete or messy data	✓
		Builds non-default cloud or distributed configs	–
WP5	Use of Modern Tools	Builds custom security mechanisms	✓
		Creates new database techniques	–
		Proposes design/coding standards	✓
		Improves existing algorithms	✓
		Defines new performance benchmarks	✓
		Explains modifications to standard workflows	✓
		Builds custom testing/validation tools	✓
WP6	Nature of Problem: Uncertainty & Ambiguity	Collects requirements from diverse users	–
		Designs role-based access/permissions	–
		Manages conflicting stakeholder needs	✓
		Builds interfaces for each stakeholder	✓
		Uses proper modelling techniques	✓
		Ensures secure data handling	✓
		Validates system with user testing	[– / ✓]
WP7	Interdependence & Multidisciplinary Factors	Breaks system into layers/modules	✓
		Integrates multiple interacting modules	✓
		Connects hardware/software components	–

WP Code	Description	Competencies	Applicable
		Handles advanced constraints	✓
		Uses DevOps/automation workflows	–
		Ensures performance & reliability	✓
		Enables cross-technology integration	✓

Table 8.10: Summary of WP competencies matched

WP Code	No. of Competencies	Level
WP1	6 of 7	High
WP2	7 of 7	High
WP3	5 of 7	High
WP4	5 of 6	High
WP5	6 of 7	High
WP6	4 of 7 [(5 of 7 once user testing is completed)]	Moderate
WP7	5 of 7	High

Note: Scale for mapping Complex Engineering Project

- Low: 1–2 competencies matched; Moderate: 3–4; High: 5 or more.
- A project is considered complex if Condition A (at least 3 of WP1–WP7 are High) and Condition B (at least 5 of WP1–WP7 are Moderate or High) are both met.

Evidence for the mapping (summary).

- **WP1:** Benjamini–Hochberg correction, a model-based multivariate leakage probe, mutual-information estimation, stratified cross-validation and entropy-based balance; nested result structures over pandas DataFrames; knowledge of statistics, machine learning, web engineering and security; references to ISO/IEC 25012, OWASP and peer-reviewed literature. No dedicated algorithmic optimisation is claimed.
- **WP2:** size and row caps and per-provider timeouts trade completeness of imports for availability; the leakage probe uses one split instead of CV to halve training cost; an HTML report was chosen over an untested PDF library; no cell values are sent to the LLM (privacy); findings are worded as "potential" and imbalance is never presented as a fairness claim (ethics).
- **WP3:** original DQI with proportional weight redistribution; the leakage-aware second

baseline run; a structural dataset-similarity score; a privacy-preserving schema summary with optional hashed names. No new ML model is proposed.

- **WP4:** quality assessment of arbitrary, messy tabular files (placeholder tokens, mixed types, casing variants, sentinel codes); BH, Dice-based name matching and the SSRF guard implemented from scratch; LLM-assisted retrieval and re-ranking. No cloud or distributed deployment is claimed.
- **WP5:** the `safe_http` guard; documented formulas and module conventions; correlation-based detectors improved with multiple-comparisons control; defect-injection benchmark datasets A–H and a degradation harness; documented changes to the standard train/evaluate workflow. No database technique is claimed.
- **WP6:** conflicting needs (LLM usefulness versus privacy; a single score versus honest uncertainty) resolved explicitly; a web UI for analysts, a REST API for developers and an exportable report for reviewers; UML and DFD models; secure handling of keys and data. Requirements were derived from the specification and literature rather than collected from many users, and the system has no roles. [Update "Validates system with user testing" after the user feedback in Table 7.12 is collected.]
- **WP7:** layered architecture with eight pipeline stages; integration of Python data-science libraries, a React frontend, six public catalogues and three LLM providers; strict constraints on outbound requests; measured performance and 141 automated tests. No hardware integration or CI/CD pipeline is claimed.

Conclusion: six attributes are rated High and one Moderate; therefore Condition A (at least 3 High) and Condition B (at least 5 Moderate or High) are both satisfied, and the project is considered a Complex Engineering Project.

APPENDIX C

Sustainable Development Goals Addressed

Table 8.11: Mapping of the project with UN Sustainable Development Goals

SDG Goal	Target	Justification / mapping	Expected impact	Level
SDG 4 – Quality Education	4.4: Increase the number of youth and adults with relevant technical skills	Synthetic defect datasets, explained formulas and live model evidence teach data-centric ML practice; usable without programming	Stronger data literacy among students and analysts	High
SDG 9 – Industry, Innovation and Infrastructure	9.5: Enhance scientific research and upgrade technological capabilities	Detects leakage and data defects that invalidate research results; free, open, laptop-scale tool	More reliable ML research and products	High
SDG 17 – Partnerships for the Goals	17.18: Increase the availability of high-quality, timely and reliable data	Quantifies and improves the quality of datasets, including public World Bank and OGD India data	Better-assessed open data for decision-making	High
SDG 16 – Peace, Justice and Strong Institutions	16.6: Develop effective, accountable and transparent institutions	Exportable, explainable quality reports document the evidence behind data-driven decisions	More transparent use of data in public bodies	Moderate
SDG 8 – Decent Work and Economic Growth	8.2: Achieve higher productivity through technological upgrading and innovation	Automates repetitive data checks and cleaning	Less time spent on manual data preparation	Moderate

Project Guide,
[Prof. GUIDE NAME]
[Assistant Professor]
Dept. of CSE

APPENDIX D

Research Publications/Patent

PAGES TO BE INSERTED

Research paper based on AutoDQ (title, authors, venue/status) and/or patent filing details.

[Insert the paper pages here, e.g. with `\includepdf{paper.pdf}` from the `pdfpages` package.]

APPENDIX E

Plagiarism Report (Similarity Report and AI Detection)

The pages that follow contain the plagiarism check of this major project report: the similarity report and the AI-writing detection report.

PAGES TO BE INSERTED

Turnitin (or institution-approved tool) similarity report and AI-detection report.

APPENDIX F
API Examples

The responses below were captured from the running application on `demo_customer_churn.csv` and shortened (`...`).

Uploading a dataset

```
POST /api/datasets/upload        (multipart/form-data, file=
    demo_customer_churn.csv)

200 OK
{ "dataset_id": "6bb42ba69736", "filename": "demo_customer_churn.csv",
  "n_rows": 2120, "n_columns": 10, "warnings": [],
  "columns": [ { "name": "customer_id", "dtype": "numeric", "n_unique":
    2000, "n_missing": 0 },
                { "name": "age", "dtype": "numeric", "n_unique": 65, "
    n_missing": 0 }, ... ] }
```

Analysing a dataset

```
POST /api/datasets/6bb42ba69736/analyze
{ "target_column": "churn", "domain": "general" }

200 OK   (excerpt: "dqi")
{ "dqi_0_1": 0.9719, "dqi_0_100": 97.2, "interpretation": "High",
  "domain_preset": "general",
  "weights_used": { "completeness": 0.25, "consistency": 0.2, "validity":
    0.2,
                    "uniqueness": 0.15, "balance": 0.2 },
  "components": [
    { "name": "completeness", "score": 0.9788, "weight": 0.25,
      "effective_weight": 0.25, "contribution": 0.2447, "available": true
    },
     ...
    { "name": "balance", "score": 0.8903, "weight": 0.2,
      "effective_weight": 0.2, "contribution": 0.1781, "available": true }
     ] }

(excerpt: "issues"[0])
{ "issue_type": "missing_values", "severity": "medium",
  "columns": ["monthly_charges"], "measured_value": 8.11,
  "explanation": "'monthly_charges' contains 8.11% missing values." }
```

Generating a corrected dataset

```
POST /api/datasets/{id}/clean

200 OK
```

```
{ "cleaned_dataset_id": "eeb98fe9f5ae",
  "cleaned_filename": "demo_customer_churn_cleaned.csv",
  "fix_log": [ "'monthly_charges': filled 172 missing value(s) with the
   median (64.645).",
               "'contract_type': filled 106 missing value(s) with the most
    frequent value
                ('Month-to-month').", ... ],
  "before": { "n_rows": 2120, "dqi": { "dqi_0_100": 97.2, ... } },
  "after":  { "n_rows": ..., "dqi": { ... } },
  "download_url": "/api/datasets/eeb98fe9f5ae/download" }
```

Structured errors

```
400 Bad Request   {"detail": "Unsupported file type '.txt'. Supported
   types: CSV, XLSX, JSON."}
404 Not Found     {"detail": "Dataset not found."}
409 Conflict      {"detail": "Dataset has not been analyzed yet. Call /
   analyze first."}
422 Unprocessable {"detail": "Unknown task 'xyz'."}
```

Provider status (no secrets returned)

```
GET /api/discovery/providers
[ { "name": "huggingface", "label": "Hugging Face", "available": true, "
   configured": true },
  { "name": "kaggle", "label": "Kaggle", "available": true, "configured":
   false,
    "hint": "Set KAGGLE_USERNAME and KAGGLE_KEY in backend/.env to enable
   Kaggle." },
  { "name": "worldbank", "label": "World Bank", "available": true, "
   configured": true,
    "catalog": { "state": "idle", "count": 0, "last_synced": null } }, ...
    ]
GET /api/llm/status   ->  { "configured": false, "provider": null }
```

APPENDIX G

Local Development Setup

All optional keys are read from a git-ignored `backend/.env` file; every setting has a default, so the application also runs without it. Commands are for Windows PowerShell; on Linux or macOS use `sourcevenv/bin/activate` and `cp` instead.

Backend

```
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -r requirements-discovery.txt     # optional: Datasets/Models
    tabs, Similar Datasets
Copy-Item .env.example .env                   # optional
python -m uvicorn app.main:app --reload       # http://localhost:8000  (
    docs at /docs)
python -m pytest tests -v                     # 141 tests
```

Frontend

```
cd frontend
npm install
Copy-Item .env.example .env                   # optional: VITE_API_URL=
    http://localhost:8000
npm run dev                                   # http://localhost:5173
```

Optional settings (`backend/.env`)

```
ALLOWED_ORIGINS=http://localhost:5173        # restrict CORS before any
    shared deployment
MAX_FILE_SIZE_MB=50
HF_TOKEN=            GITHUB_TOKEN=           # higher rate limits only
KAGGLE_USERNAME=     KAGGLE_KEY=             # enables Kaggle
OGD_INDIA_API_KEY=                           # enables OGD India
LLM_PROVIDER=groq    GROQ_API_KEY=           # Similar Datasets AI
    assistance
LLM_FALLBACK_PROVIDER=gemini   GEMINI_API_KEY=
LLM_SEND_COLUMN_NAMES=true                   # false = hash column names
    in prompts
```

Regenerating data and running the harness

```
cd datasets
python generate_datasets.py
python validation_harness.py A_clean.csv target --levels 5
python validation_harness.py demo_customer_churn.csv churn --levels 5
```

APPENDIX H
Security Controls Checklist

Table 8.12: Mapping of OWASP API Security Top 10 (2023) risks to AutoDQ controls

OWASP API risk	Control in AutoDQ
API1 Broken Object Level Authorization	Datasets addressed by random 12-hex-character identifiers. AutoDQ is a single-user tool without accounts; per-user authorisation is future work and is required before multi-user deployment.
API2 Broken Authentication	Not applicable in the current single-user scope (no accounts); the API should be bound to localhost or placed behind an authenticating proxy when shared.
API3 Broken Object Property Level Authorization	Responses built from explicit result objects; no endpoint accepts arbitrary object properties; provider and LLM status never include keys.
API4 Unrestricted Resource Consumption	50 MB upload and import cap enforced while streaming; 200,000-row cap on paged imports; query length, page and limit bounds; per-provider timeouts; search-result cache.
API5 Broken Function Level Authorization	No privileged functions exposed; configuration only through server-side environment variables.
API6 Unrestricted Access to Sensitive Business Flows	Expensive catalogue syncs run once in the background and are cached; LLM calls limited in count and retries.
API7 Server-Side Request Forgery	`safe_http`: HTTPS only, port 443, host allow-list, no IP literals, public addresses only, manual redirect re-validation, credentials only to the original host.
API8 Security Misconfiguration	Configurable CORS origins (restrict before deployment); TLS verification never disabled; short error messages without stack traces or URLs.
API9 Improper Inventory Management	Single documented endpoint list (Table 6.5); OpenAPI schema at `/docs`; versioned dependency requirements.
API10 Unsafe Consumption of APIs	Catalogue and LLM responses size-capped and schema-validated; provider identifiers validated before use in URLs; LLM output filtered to known candidate ids; remote text rendered as escaped plain text.